

The cover features a decorative border of pink cherry blossoms. A large, dense cluster of blossoms is in the top-left corner, a smaller branch is in the top-right, and a wide border of blossoms runs along the bottom and left sides.

2026

MODUL PRAKTIKUM 4
BASIC DIGITAL AUDIO PROCESSING
OPTIMIZED VERSION

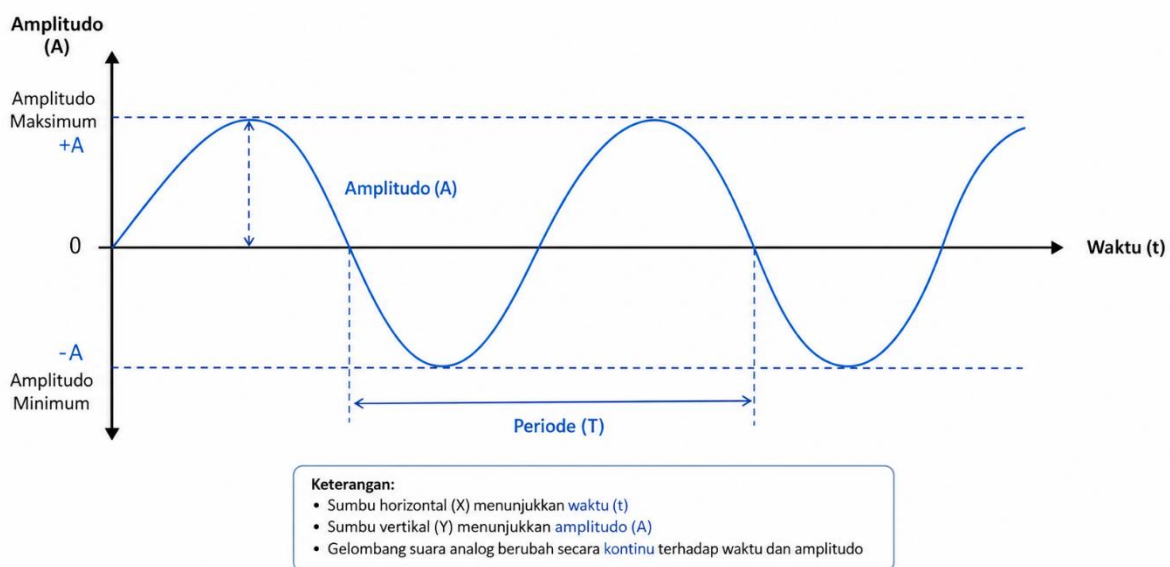
Disusun Oleh:
Ryan F F Hakim

Dipersembahkan Untuk:
Sains Data

Mata Kuliah
Sistem Multimedia

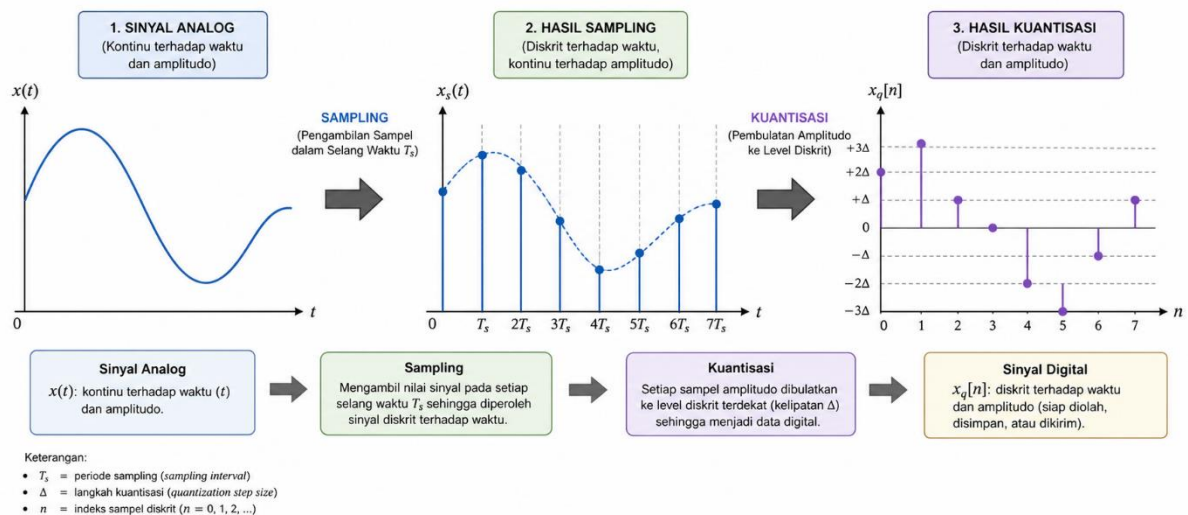
PENDAHULUAN

Suara pada dasarnya merupakan gelombang mekanik yang merambat melalui medium fisik seperti udara akibat adanya variasi tekanan udara yang berkelanjutan. Dalam bentuk aslinya, sinyal suara bersifat kontinu atau analog, di mana setiap perubahan sekecil apa pun pada waktu dan amplitudo memiliki resolusi nilai yang tak terhingga. Pemahaman mendalam mengenai karakteristik fisik gelombang suara—terutama frekuensi yang menentukan nada atau *pitch*, serta amplitudo yang menentukan volume atau tingkat tekanan suara—merupakan landasan utama yang wajib dikuasai sebelum memasuki ranah komputasi. Tanpa adanya pemahaman terhadap sifat fisis ini, manipulasi data pada tahapan digital akan kehilangan konteks akustiknya.



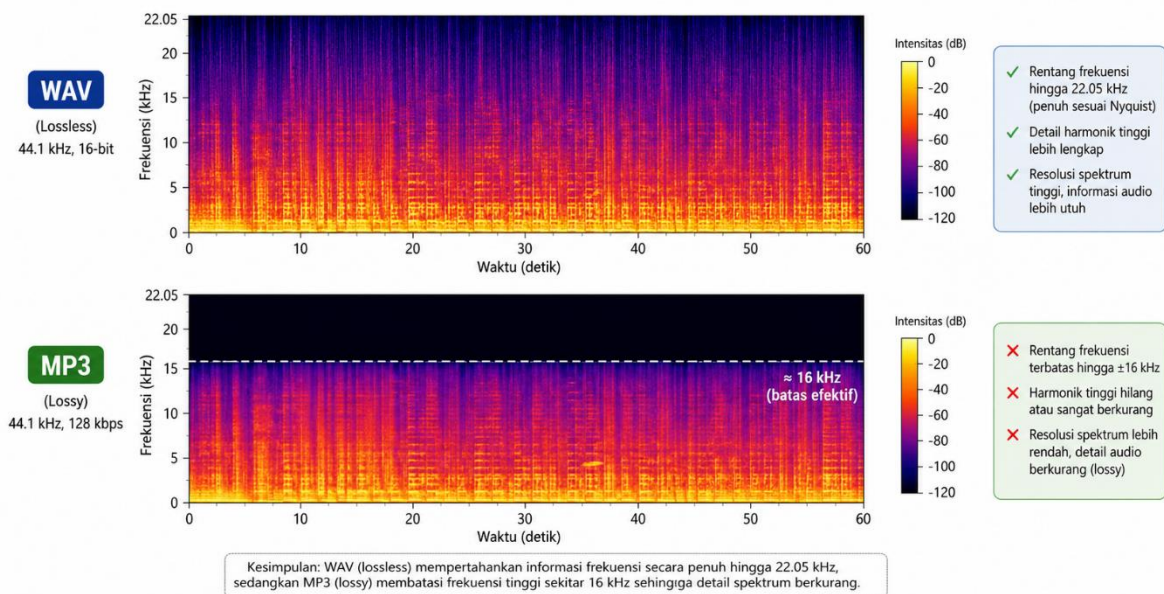
Gambar 1.1: Representasi Visual Gelombang Suara Analog Berdasarkan Waktu dan Amplitudo

Agar gelombang suara analog dapat diterima, diolah, dan dimanipulasi oleh sistem komputer, diperlukan sebuah proses konversi ke dalam format digital melalui tahap digitalisasi (*Analog-to-Digital Conversion*). Tahapan ini melibatkan dua mekanisme utama, yakni **pencuplikan** (*sampling*) dan **kuantisasi** (*quantization*). Proses *sampling* bertugas mengambil nilai amplitudo gelombang analog pada interval waktu yang sangat singkat dan teratur, sedangkan proses *quantization* berfungsi membulatkan nilai amplitudo hasil cuplikan tersebut ke dalam tingkat nilai diskrit yang dapat direpresentasikan oleh bit biner komputer. Kualitas dan akurasi sinyal digital yang dihasilkan sangat bergantung pada tingginya *sample rate* dan *bit depth* yang digunakan saat proses konversi berlangsung.



Gambar 1.2: Skema Konversi Sinyal Analog ke Digital melalui Tahapan Sampling dan Kuantisasi

Setelah sinyal suara berhasil direpresentasikan sebagai kumpulan data numerik digital, data tersebut kemudian disimpan menggunakan berbagai format berkas (*file*) audio yang memiliki karakteristik spesifik. Format audio tanpa kompresi (*uncompressed*), seperti WAV atau AIFF, menyimpan seluruh data hasil sampel mentah tanpa menghilangkan informasi apa pun, sehingga format ini menjadi standar mutlak untuk proses pengolahan audio profesional yang membutuhkan fidelitas tinggi. Di sisi lain, terdapat format terkompresi (*compressed*) seperti MP3 yang mengaplikasikan algoritma pemodelan psikoakustik untuk membuang porsi data suara yang tidak mudah ditangkap oleh batas pendengaran manusia, sehingga mampu mereduksi ukuran penyimpanan secara drastis dengan sedikit kompromi pada kualitas vokal atau instrumen.



Gambar 1.3: Perbandingan Resolusi Spektrum Audio antara Format WAV dan Format MP3

Pemrosesan audio digital (*Digital Audio Processing*) dasar mencakup berbagai teknik manipulasi matematis yang diterapkan secara langsung pada sekumpulan data sampel audio digital tersebut guna mencapai luaran suara yang diinginkan. Teknik-teknik fundamental ini meliputi penyesuaian amplitudo keseluruhan (*gain staging* atau *normalization*), modifikasi spektrum frekuensi (*equalization* atau *filtering*), hingga pencampuran beberapa sinyal audio menjadi satu kesatuan (*mixing*). Seluruh manipulasi matematis ini difasilitasi melalui algoritma Pemrosesan Sinyal Digital (*Digital Signal Processing* / *DSP*) yang diimplementasikan di dalam bahasa pemrograman maupun perangkat lunak, memungkinkan perubahan karakteristik audio secara presisi, terukur, dan dapat diulang (*repeatable*).



Gambar 1.4: Tampilan Antarmuka Perangkat Lunak Digital Audio Workstation Beserta Analisis Spektrum

Modul praktikum ini disusun secara terstruktur untuk memberikan pemahaman teknis dan aplikatif mengenai konsep-konsep dasar manipulasi sinyal audio digital. Melalui serangkaian eksperimen yang telah dirancang, penerapan berbagai algoritma DSP dasar dapat diobservasi, disimulasikan, dan dianalisis parameter-parameternya secara komprehensif. Penguasaan materi serta keterampilan teknis yang diperoleh melalui penyelesaian modul ini akan berfungsi sebagai fondasi analitis yang kokoh untuk mempelajari rekayasa sinyal audio tingkat lanjut, pengenalan wicara (*speech recognition*), maupun komputasi akustik di masa mendatang.

LANDASAN TEORI

2.1 KONSEP DASAR SINYAL AUDIO DIGITAL

Sinyal audio pada dasarnya bermula dari gelombang mekanik berupa variasi tekanan udara yang merambat melalui medium fisik. Ketika gelombang suara ini ditangkap oleh transduser (seperti mikrofon), fluktuasi tekanan fisik tersebut diubah menjadi sinyal elektrik kontinu (analog) yang sebanding dengan amplitudo suara asli. Sinyal audio digital adalah bentuk representasi diskrit dari sinyal analog ini ke dalam format bilangan biner agar dapat disimpan, dimanipulasi, dan diproses oleh sistem komputasi.

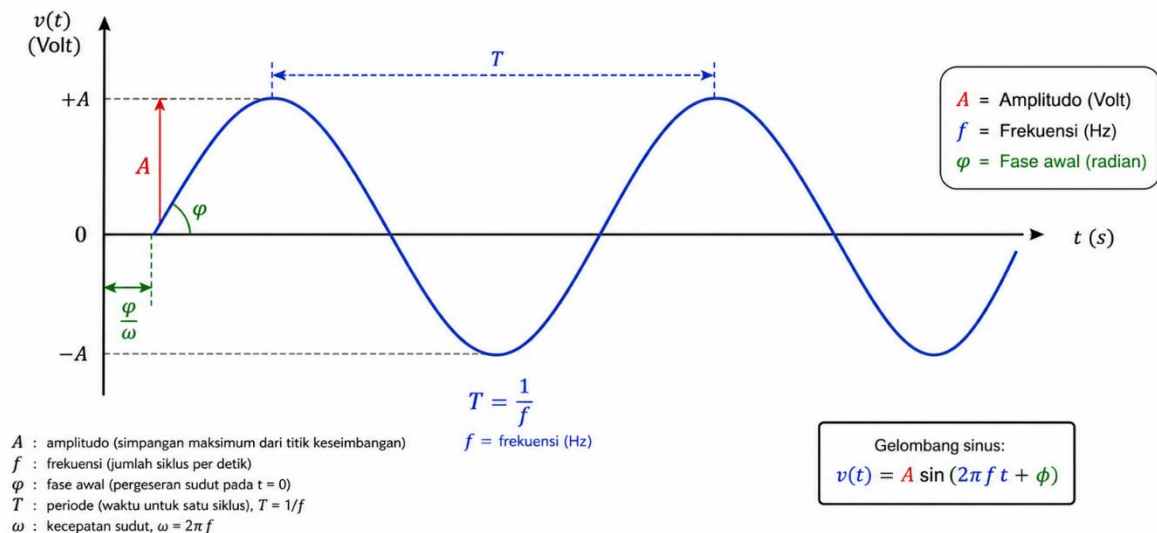
2.1.1 KOMPONEN PEMBENTUK SINYAL AUDIO

Secara matematis dan fisis, persamaan dasar gelombang suara analog dapat dideskripsikan sebagai:

$$y(t) = A \cdot \sin(2\pi ft + \phi)$$

Di mana komponen-komponen utamanya meliputi:

- **Amplitudo (A):** Merupakan ukuran perpindahan maksimum gelombang dari titik kesetimbangannya. Dalam konteks persepsi manusia, amplitudo berkorelasi dengan tingkat tekanan suara (*Sound Pressure Level / SPL*) atau kekerasan suara (*loudness*).
- **Frekuensi:** Merupakan jumlah siklus gelombang yang terjadi dalam rentang waktu satu detik, diukur dalam satuan Hertz (Hz). Frekuensi mendikte persepsi tinggi-rendahnya nada (*pitch*). Telinga manusia normal umumnya mampu merespons rentang frekuensi antara 20 Hz hingga 20.000 Hz (20 kHz).
- **Fase (Phase):** Posisi titik tertentu pada satu siklus gelombang relatif terhadap titik awal nol, biasanya diukur dalam derajat atau radian. Perbedaan fase sangat memengaruhi interaksi antar gelombang (interferensi konstruktif maupun destruktif).

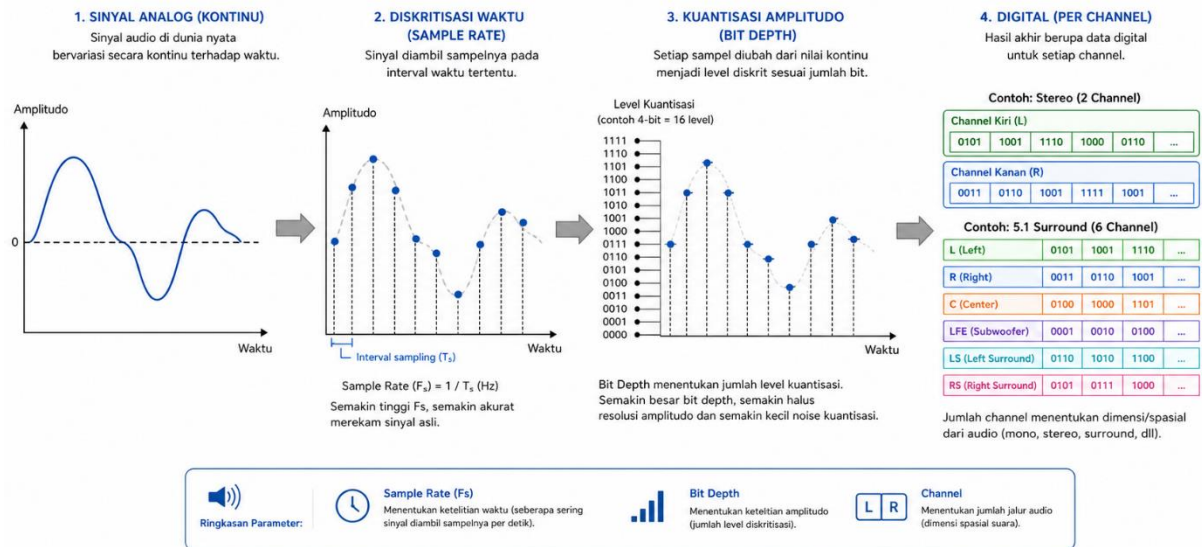


Gambar 2.1: Representasi visual gelombang sinus analog beserta parameter amplitudo, frekuensi, dan fase.

2.1.2 PARAMETER STANDAR AUDIO DIGITAL

Dalam format digital, representasi audio diikat oleh beberapa parameter teknis:

- **Sample Rate (Frekuensi Sampling):** Frekuensi pengambilan cuplikan data sinyal per detiknya. Standar yang sering digunakan mencakup 44.100 Hz (standar CD Audio) dan 48.000 Hz (standar Audio Visual/DVD).
- **Bit Depth (Kedalaman Bit):** Jumlah bit memori yang dialokasikan untuk menyimpan nilai amplitudo dari satu buah cuplikan (*sample*). Hal ini menentukan resolusi dari rentang dinamis (*dynamic range*) sinyal.
- **Saluran (Channel):** Merujuk pada jumlah lintasan audio independen yang direkam atau diputar secara bersamaan. Format mono menggunakan satu saluran tunggal, sedangkan format stereo memisahkan lintasan menjadi saluran kiri (L) dan kanan (R) untuk menciptakan ilusi ruang atau dimensi suara.



Gambar 2.2: Proses diskritisasi sinyal analog menjadi digital melalui parameter sample rate dan bit depth serta channel.

2.2 PROSES DIGITALISASI (ANALOG-TO-DIGITAL CONVERSION)

Proses konversi sinyal analog yang kontinu menjadi rentetan nilai digital yang diskrit melibatkan perangkat *Analog-to-Digital Converter* (ADC). Konversi ini mencakup dua proses esensial: Sampling (pencuplikan) dan Kuantisasi (quantization).

2.2.1 PENCUPLIKAN (SAMPLING) DAN TEOREMA NYQUIST-SHANNON

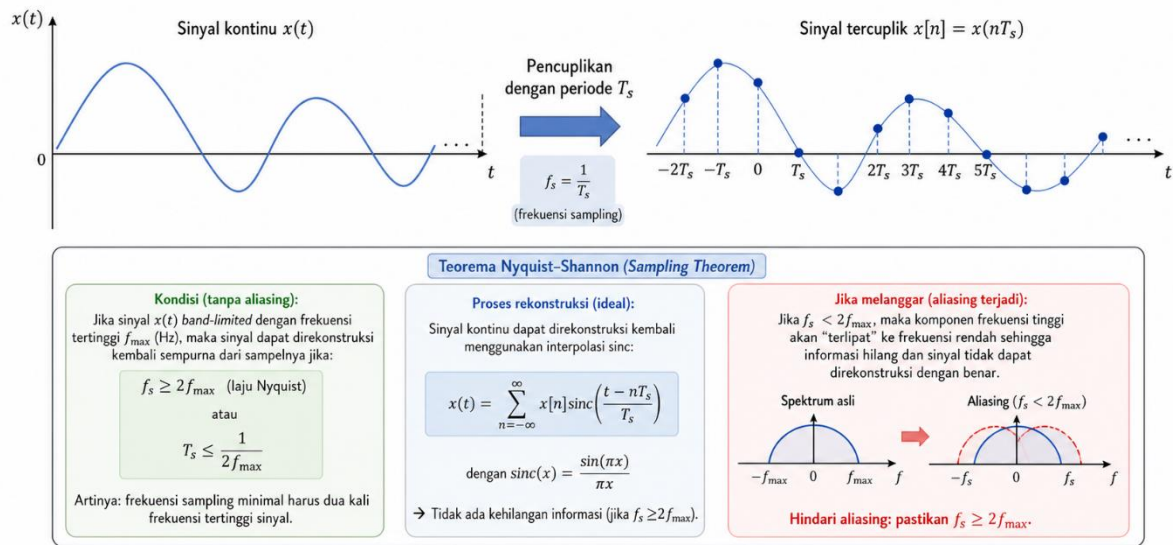
Sampling adalah proses mendiskritisasi sinyal kontinu pada domain waktu. Sinyal kontinu diukur amplitudonya pada interval waktu yang sangat singkat dan beraturan.

Batas frekuensi pencuplikan diatur oleh **Teorema Nyquist-Shannon**, yang mendikte bahwa untuk merekonstruksi suatu sinyal gelombang dengan sempurna tanpa adanya kehilangan informasi, frekuensi pencuplikan (f_s) harus minimum bernilai dua kali lipat dari komponen frekuensi tertinggi (f_{max}) yang terdapat dalam sinyal tersebut:

$$f_s \geq 2 \times f_{max}$$

Nilai $f_s/2$ sering disebut sebagai **Frekuensi Nyquist**. Jika sebuah sinyal memiliki komponen frekuensi yang melebihi Frekuensi Nyquist, maka akan terjadi fenomena cacat sinyal yang dikenal sebagai **Aliasing**. Aliasing menyebabkan frekuensi-frekuensi tinggi yang terpotong tersebut terefleksikan (terlipat) kembali ke dalam spektrum frekuensi rendah sebagai komponen *noise* buatan (harmonik palsu)

yang tidak dapat dihilangkan. Untuk mencegah aliasing, sinyal analog selalu melewati *Anti-Aliasing Filter* (Low-Pass Filter analog) sebelum masuk ke proses pencuplikan.



Gambar 2.3: Proses pencuplikan sinyal kontinu berdasarkan Teorema Nyquist-Shannon untuk menghindari kehilangan informasi.

2.2.2 KUANTISASI (QUANTIZATION) DAN RESOLUSI AMPLITUDO

Setelah sinyal didiskritisasi terhadap waktu (sampling), nilai amplitudo pada tiap cuplikan harus dibulatkan ke tingkat nilai biner terdekat yang tersedia. Proses ini dinamakan kuantisasi.

Jumlah tingkatan (level) nilai amplitudo bergantung sepenuhnya pada **Bit Depth** (N).

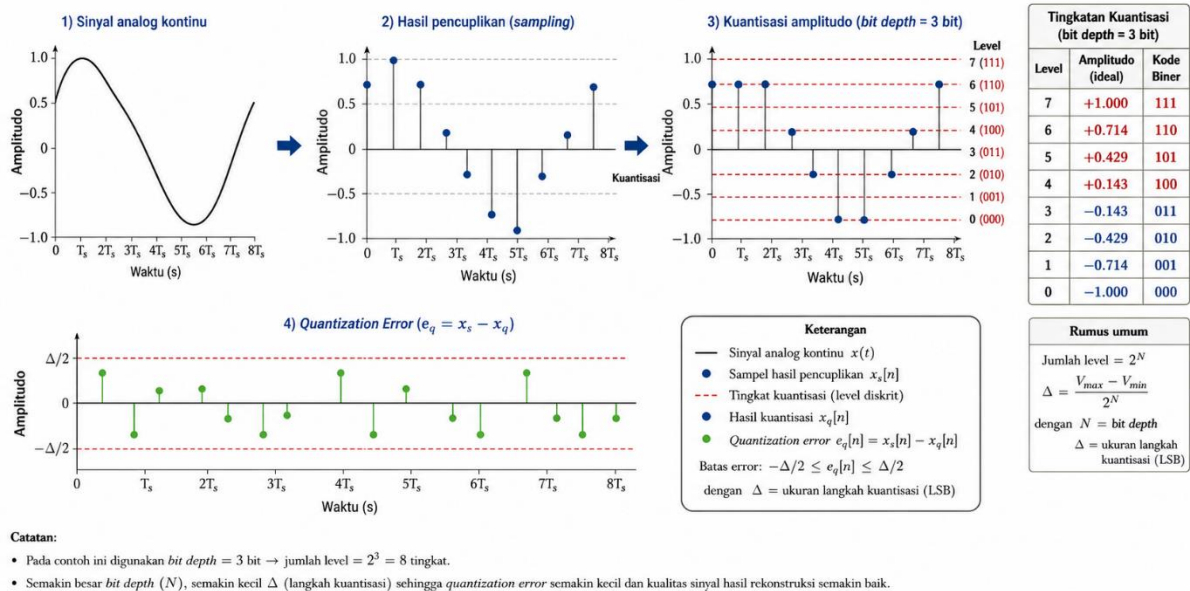
$$\text{JumlahLevel} = 2^N$$

Sebagai contoh, bit depth 8-bit hanya menyediakan 256 tingkat amplitudo (2^8), sedangkan bit depth 16-bit menyediakan 65.536 tingkatan (2^{16}). Pembulatan amplitudo sinyal kontinu ke tingkat diskrit yang paling dekat ini menghasilkan suatu selisih galat yang disebut sebagai **Quantization Error** atau *Quantization Noise*. Semakin tinggi resolusi bit, semakin kecil selisih galat pembulatan tersebut, sehingga *noise floor* (dasar kebisingan bawaan) menjadi semakin rendah.

Rasio antara amplitudo sinyal maksimal terhadap *quantization noise* dikenal sebagai *Signal-to-Noise Ratio* (SNR) teoretis. Secara matematis, batas maksimal SNR berdasarkan bit depth (N) dirumuskan sebagai:

$$SNR_{dB} \approx 6.02 \cdot N + 1.76$$

Di mana setiap kenaikan 1-bit kedalaman memori akan menyumbangkan peningkatan ruang dinamis (*Dynamic Range*) sekitar **6.02 dB**. Dengan demikian, format audio 16-bit memiliki rentang dinamis maksimal sekitar 96 dB, sedangkan format 24-bit mencapai sekitar 144 dB.

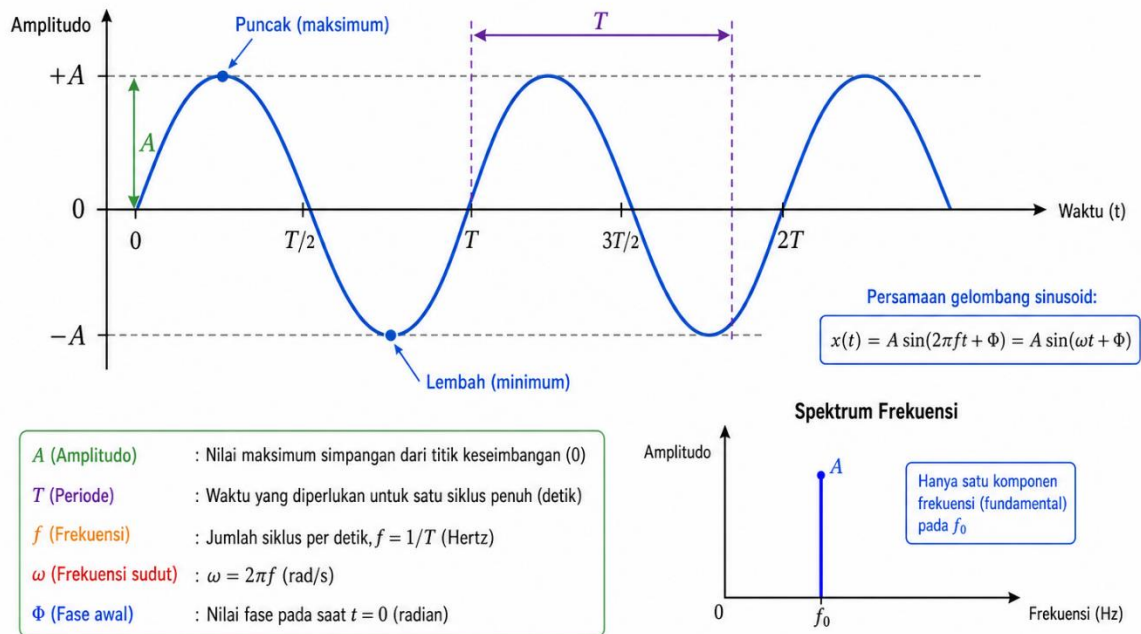


Gambar 2.4: Proses kuantisasi amplitudo sinyal hasil pencuplikan ke dalam tingkatan nilai biner diskrit berdasarkan bit depth serta visualisasi quantization error.

2.3 KARAKTERISTIK GELOMBANG DASAR (PRIMITIVE WAVEFORMS)

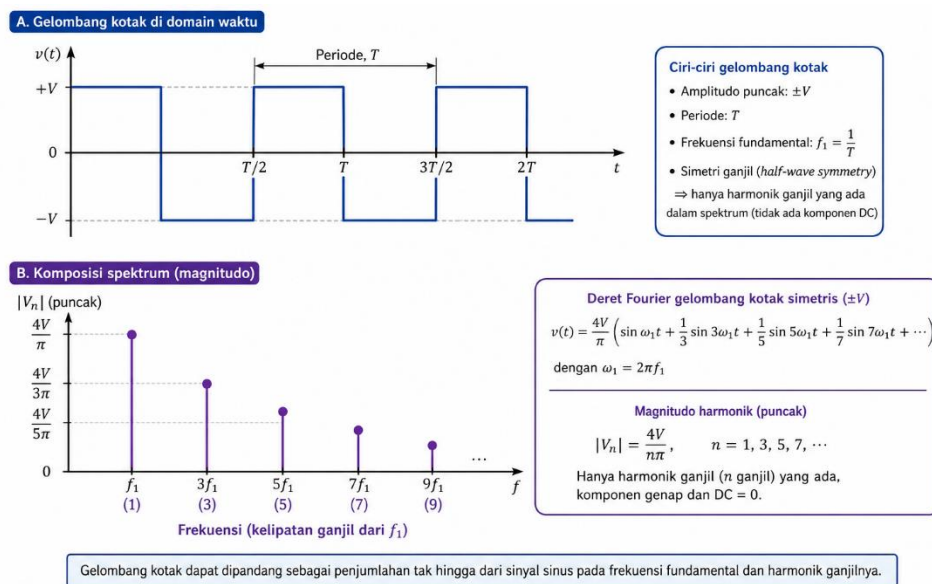
Gelombang audio mendasar (*primitive waveforms*) merupakan bentuk sinyal paling sederhana yang menjadi balok penyusun untuk bentuk suara yang lebih kompleks. Secara harmonik, gelombang dapat dijabarkan menggunakan deret Fourier.

- 1. Gelombang Sinusoid (Sine Wave):** Gelombang paling murni yang energi frekuensinya secara terpusat hanya berada pada satu frekuensi fundamental tunggal, tanpa adanya overtone atau harmonik tambahan. Sinyal ini terdengar sangat halus di telinga manusia.



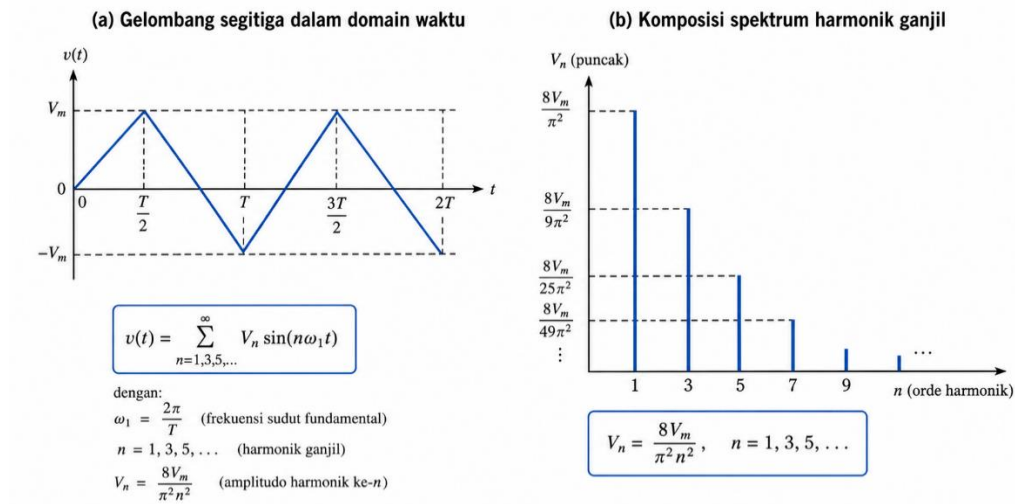
Gambar 2.5: Karakteristik visual gelombang sinusoid sebagai bentuk sinyal paling murni yang hanya memiliki satu frekuensi fundamental tunggal.

2. **Gelombang Kotak (Square Wave):** Sinyal transisi seketika antara nilai minimum dan maksimum amplitudo yang konstan. Secara spektrum Fourier, gelombang kotak tersusun dari frekuensi fundamental ditambah seluruh deret **harmonik ganjil** ($3f$, $5f$, $7f$, dst) yang amplitudonya menurun sebanding dengan $1/n$. Menghasilkan suara yang keras dan berdengung tajam.



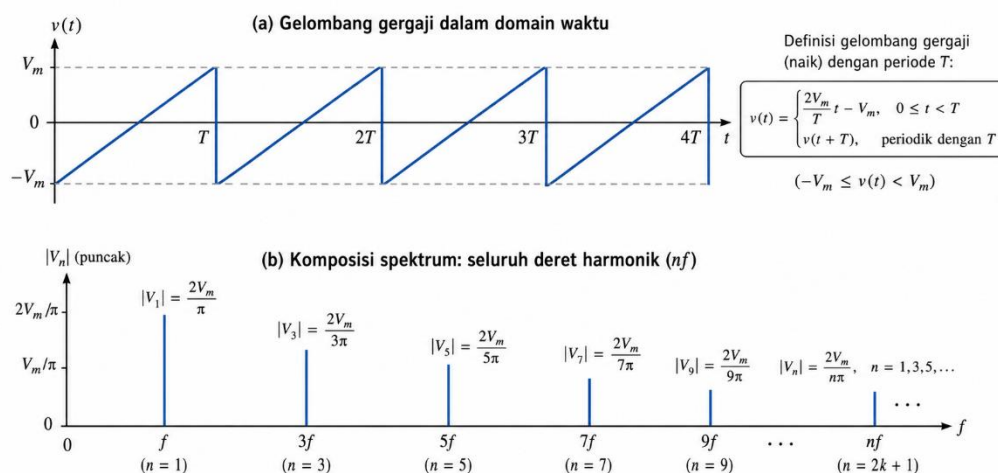
Gambar 2.6: Karakteristik visual gelombang kotak dan komposisi spektrumnya yang terdiri dari frekuensi fundamental serta deret harmonik ganjil.

3. **Gelombang Segitiga (Triangle Wave):** Mengandung komponen frekuensi dasar dan frekuensi **harmonik ganjil**, sama seperti gelombang kotak, tetapi laju penurunan amplitudo harmoniknya jauh lebih curam ($1/n^2$). Karakteristik suaranya berada di tengah-tengah antara kejernihan gelombang sinus dan kekasaran gelombang kotak.



Gambar 2.7: Bentuk gelombang segitiga dalam domain waktu dan komposisi spektrum harmonik ganjilnya.

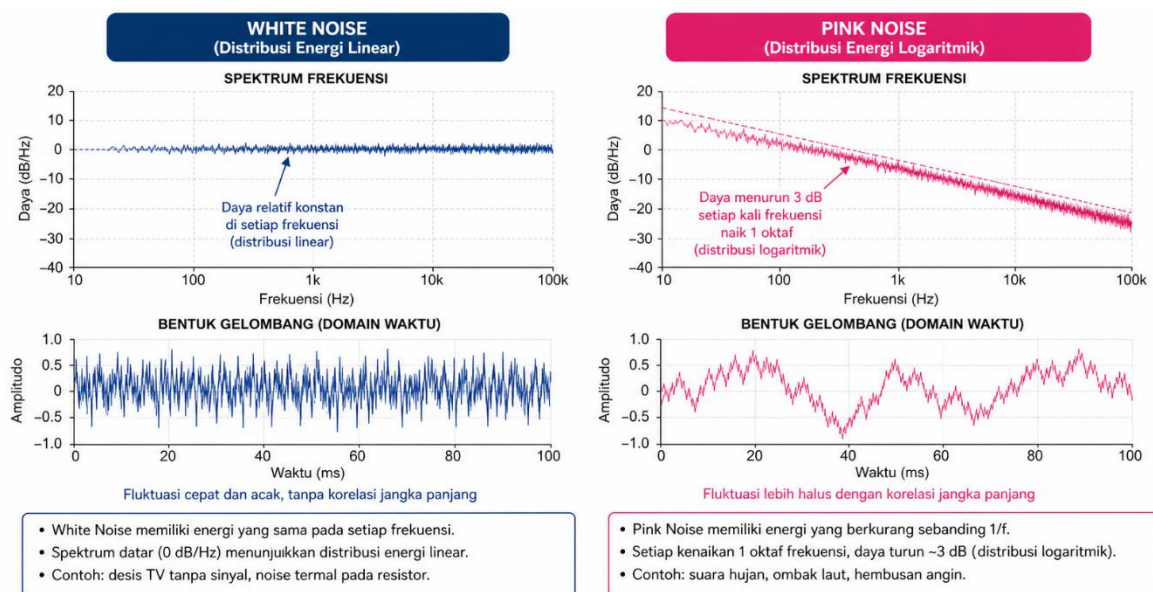
4. **Gelombang Gergaji (Sawtooth Wave):** Sinyal yang terus naik secara linier dan jatuh seketika secara periodik. Gelombang ini sangat kaya secara spektral karena memuat energi frekuensi fundamental beserta seluruh deret **harmonik ganjil dan genap**. Banyak diaplikasikan sebagai sumber suara dasar pada instrumen *synthesizer*.



Gambar 2.8: Bentuk gelombang gergaji dalam domain waktu dan komposisi spektrumnya yang mencakup seluruh deret harmonik (nf).

5. **Derau (Noise):** Sinyal stokastik (acak) tak beraturan tanpa frekuensi fundamental.

- *White Noise*: Distribusi energi bersifat konstan atau setara pada setiap rentang frekuensi secara linear. Oleh telinga manusia, white noise terdengar sangat tajam (mendesis keras) karena manusia lebih sensitif terhadap frekuensi tinggi.
- *Pink Noise*: Distribusi daya spektrumnya berbanding terbalik terhadap frekuensi ($1/f$). Energi pada pita frekuensi secara logaritmik bernilai sama (energi menurun 3 dB setiap oktaf). Karakteristik *pink noise* lebih disukai dalam kalibrasi akustik karena responsnya terdengar rata di sistem pendengaran manusia.



Gambar 2.9: Perbandingan spektrum frekuensi dan bentuk gelombang antara White Noise (distribusi energi linear) dan Pink Noise (distribusi energi logaritmik).

2.4 ANALISIS DOMAIN WAKTU DAN DOMAIN FREKUENSI

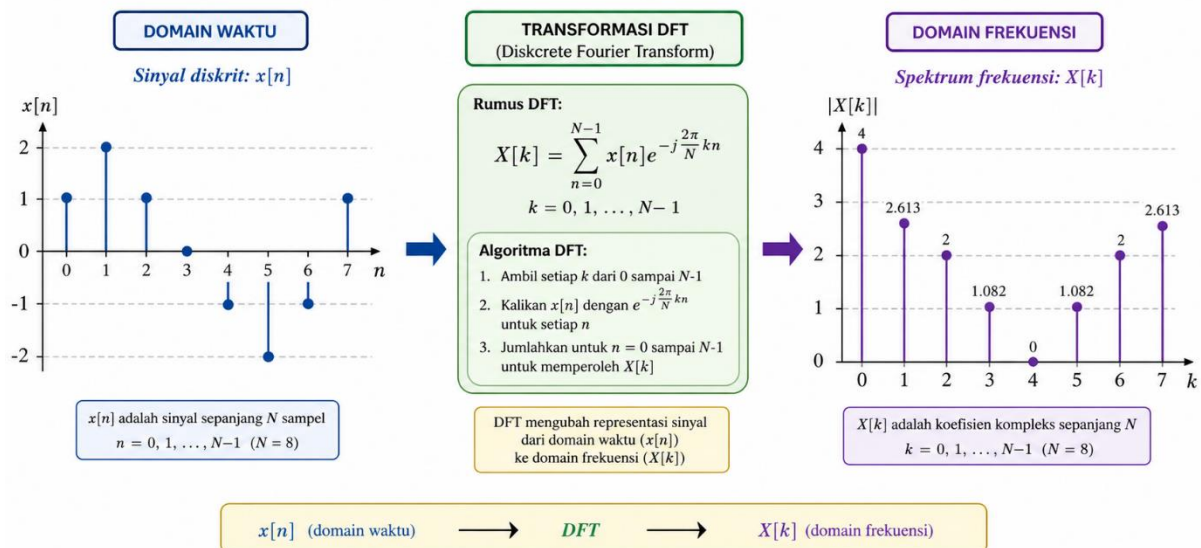
Representasi standar sinyal audio dalam wujud grafik amplitudo terhadap waktu disebut analisis pada **Domain Waktu** (*Time Domain*). Representasi ini berguna untuk melihat perubahan volume, durasi sinyal, batas *clipping*, dan *envelope* (ASDR: Attack, Sustain, Decay, Release). Namun, representasi waktu murni tidak memperlihatkan informasi struktural tentang komposisi nada atau frekuensi yang terkandung di dalam sinyal tersebut.

2.4.1 DISCRETE FOURIER TRANSFORM (DFT)

Untuk menguraikan gelombang kompleks ke dalam pita-pita frekuensi konstituennya, dilakukan transformasi ke **Domain Frekuensi**. Secara matematis, transformasi ini difasilitasi oleh algoritma *Discrete Fourier Transform* (DFT) dengan rumusan absolut:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}$$

Persamaan di atas mengkonversi deret data titik waktu ($x[n]$) menjadi deret data bilangan kompleks ($X[k]$) yang mendeskripsikan magnitudo (kekuatan) dan fase dari sekumpulan gelombang sinus dalam berbagai rentang frekuensi tertentu. Algoritma perhitungan efisien untuk mengkomputasi matriks DFT di dalam sistem pemrograman dikenal secara meluas sebagai **Fast Fourier Transform (FFT)**.

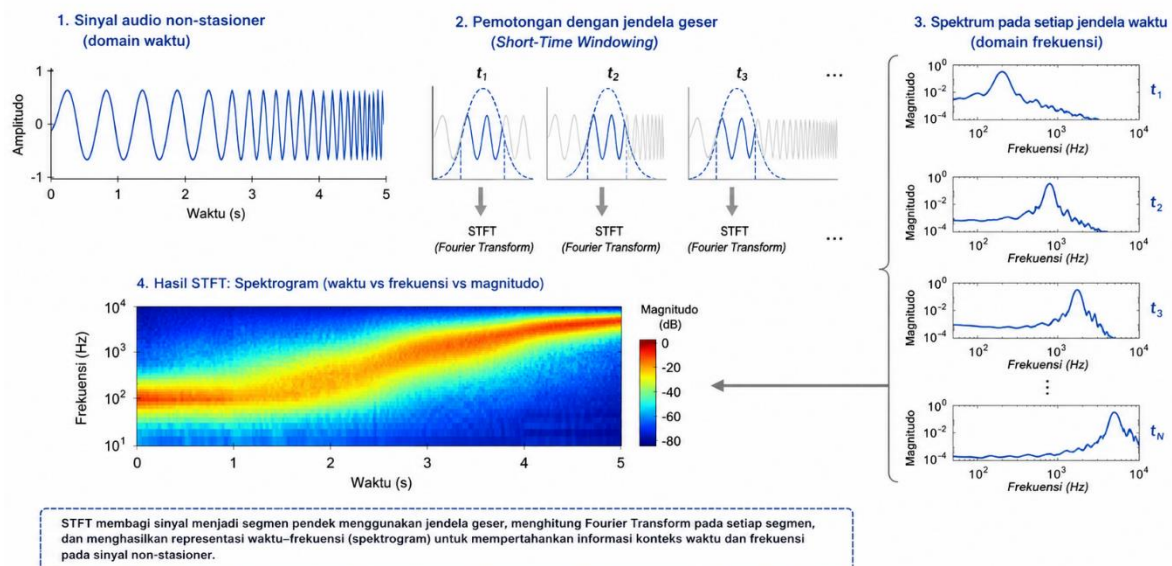


Gambar 2.10: Proses transformasi sinyal dari domain waktu ($x[n]$) ke domain frekuensi ($X[k]$) menggunakan algoritma DFT.

2.4.2 SPEKTROGRAM (SPECTROGRAM)

Meski FFT sangat tangguh untuk mendeteksi frekuensi, transformasi murni kehilangan konteks "waktu kapan" frekuensi tersebut terjadi. Untuk analisis sinyal yang bersifat non-stasioner (kandungan frekuensinya berubah seiring waktu, seperti pada sampel musik atau ucapan manusia), digunakan algoritma *Short-Time Fourier Transform* (STFT).

STFT mengiris sinyal panjang menjadi jendela-jendela waktu pendek (*windowing*) secara *overlapping*, lalu melakukan FFT pada masing-masing jendela tersebut. Hasil gabungan matriks analisis 3 dimensi ini divisualisasikan dalam bentuk **Spektrogram** (Sumbu X merepresentasikan Waktu, Sumbu Y merepresentasikan Frekuensi, dan Gradasi Warna/Intensitas merepresentasikan Magnitudo energi frekuensi tersebut dalam satuan dB).



Gambar 2.11: Analisis sinyal audio non-stasioner menggunakan Short-Time Fourier Transform (STFT) untuk mempertahankan konteks waktu dan frekuensi.

2.5 PEMROSESAN SINYAL AUDIO DASAR

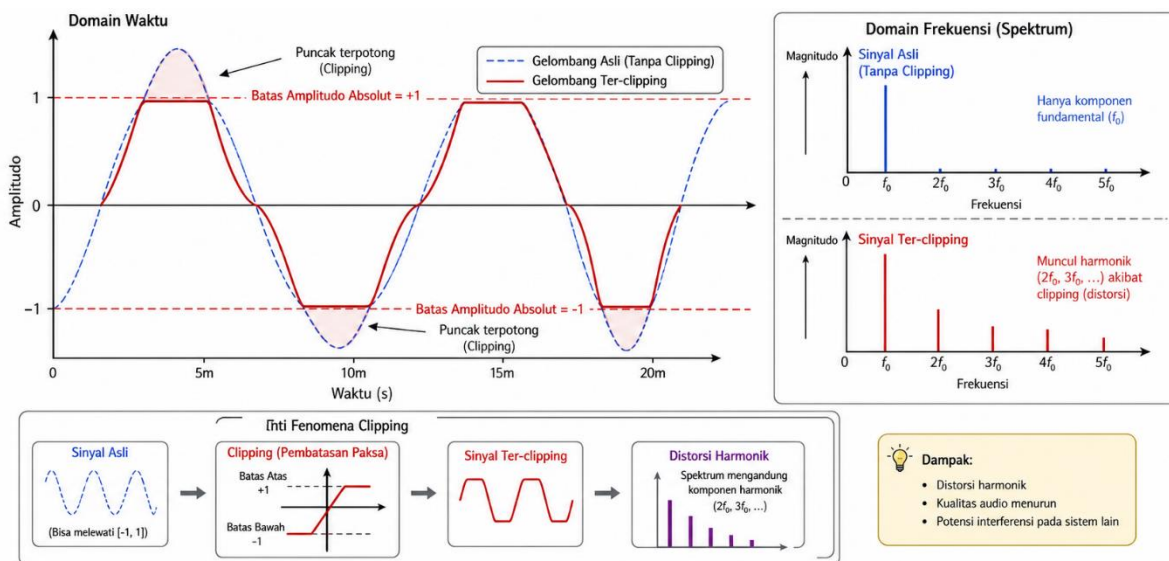
2.5.1 SKALABILITAS AMPLITUDO (VOLUME DAN GAIN)

Operasi paling fundamental dalam pemrosesan audio adalah memodifikasi tingkat amplitudo keseluruhan sinyal. Secara matematis, hal ini dilakukan melalui perkalian matriks sinyal asli dengan konstanta skalar linear penguatan (*Gain*). Dalam praktiknya, manipulasi volume direpresentasikan menggunakan skala logaritmik (Desibel / dB) berdasarkan persamaan perbandingan amplitudo:

$$Gain_{dB} = 20 \log_{10} \left(\frac{A_{baru}}{A_{lama}} \right)$$

Pemodelan desibel ini teramat krusial karena persepsi sensitivitas pendengaran biologis manusia pada dasarnya beroperasi secara logaritmik, bukan linear.

Jika nilai sinyal melewati batas kapasitas amplitudo absolut (umumnya terdefinisi pada interval absolut ± 1.0 dalam standar representasi *floating-point* digital), akan terjadi distorsi yang disebut **Clipping**. *Clipping* memotong puncak gelombang secara paksa menjadi garis datar, yang merusak struktur harmonik asli dan menginjeksikan derau frekuensi tinggi yang menyerupai karakteristik gelombang kotak tiruan.



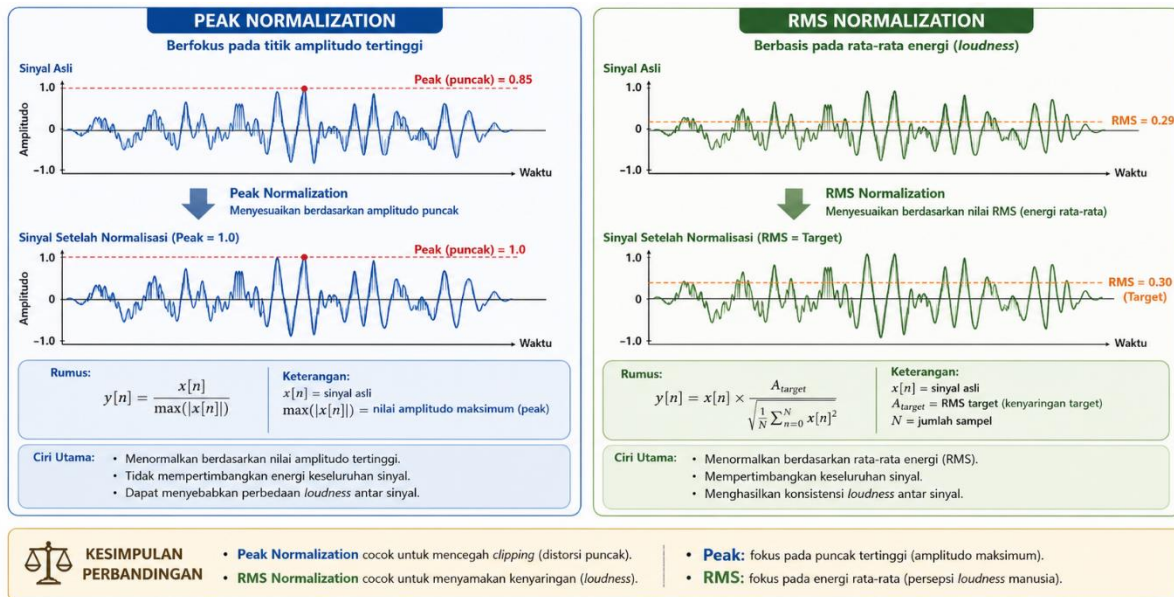
Gambar 2.12: Representasi fenomena clipping pada domain digital, di mana puncak gelombang yang melewati batas amplitudo absolut $[-1,1]$ dipotong secara paksa, mengakibatkan distorsi harmonik.

2.5.2 NORMALISASI SINYAL

Normalisasi merupakan proses penyesuaian rasio amplitudo keseluruhan sinyal untuk mencapai suatu target level acuan secara seragam, tanpa mendistorsi rasio relatif antar komponen frekuensi aslinya.

- **Peak Normalization:** Sinyal diskalakan dengan suatu konstanta identik sedemikian rupa hingga sampel dengan magnitudo puncak (*peak*) absolut menyentuh angka maksimal yang diizinkan (misalnya, dipatok pada 0 dBFS atau skala amplitudo $\$1.0\$$).
- **RMS Normalization:** Sinyal diskalakan dengan berlandaskan pada energi rata-rata keseluruhan (RMS - *Root Mean Square*), bukan sekadar pada satu

titik puncak sesaat. Pendekatan RMS secara komputasional terbukti lebih berkorelasi secara langsung dengan level kenyaringan subyektif (*perceived loudness*) yang dirasakan oleh telinga telanjang.

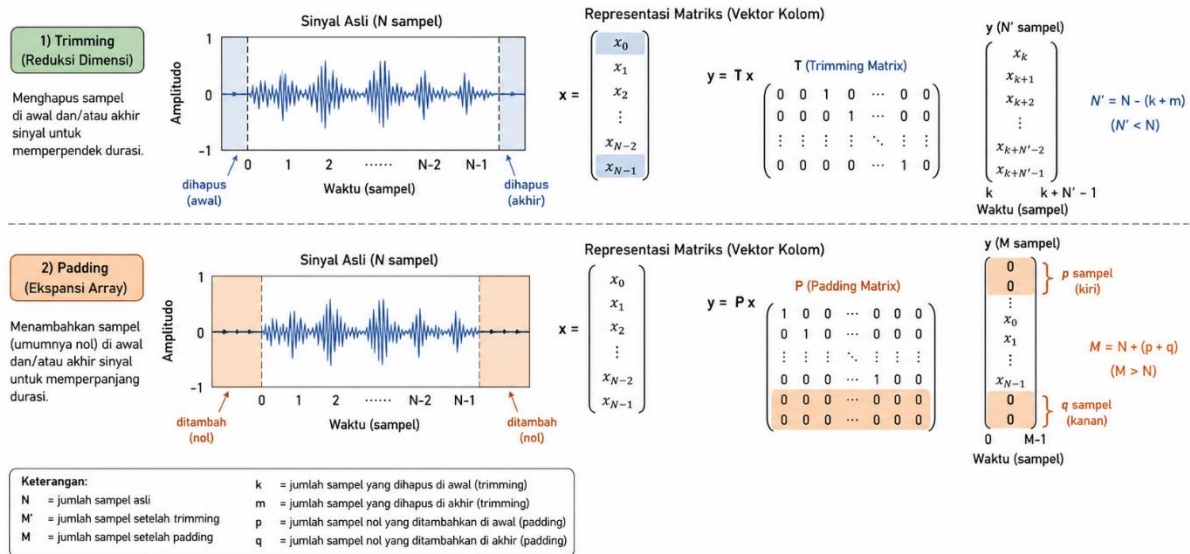


Gambar 2.13: Perbandingan metode normalisasi sinyal: Peak Normalization yang berfokus pada titik amplitudo tertinggi dan RMS Normalization yang berbasis pada rata-rata energi untuk mencapai konsistensi kenyaringan (*loudness*).

2.5.3 MODIFIKASI LINIMASA WAKTU (TRIMMING & PADDING)

Ditinjau dari struktur datanya, sinyal audio adalah *array* berdimensi tunggal. Modifikasi durasi secara literal direalisasikan melalui manipulasi matriks linear.

- **Trimming:** Eliminasi irisan indeks *array* di blok durasi tertentu. Dalam subdisiplin algoritma reduksi keheningan (*Silence Removal*), deteksi kalkulasi batas energi diaplikasikan di setiap partisi *frame* berurutan; *frame* dengan energi terakumulasi di bawah ambang batas (*threshold* dalam metrik dB) direduksi secara otomatis dari keseluruhan dimensi.
- **Padding:** Pengikutsertaan elemen nilai konstan bernilai nol (*Zero Padding*) ke dalam *array* matriks sinyal. Representasi matematis deret nol diterjemahkan secara fisis sebagai absennya perubahan getaran tekanan, yang dieksekusi oleh penyambung analog (DAC) murni sebagai bentuk keheningan akustik absolut.



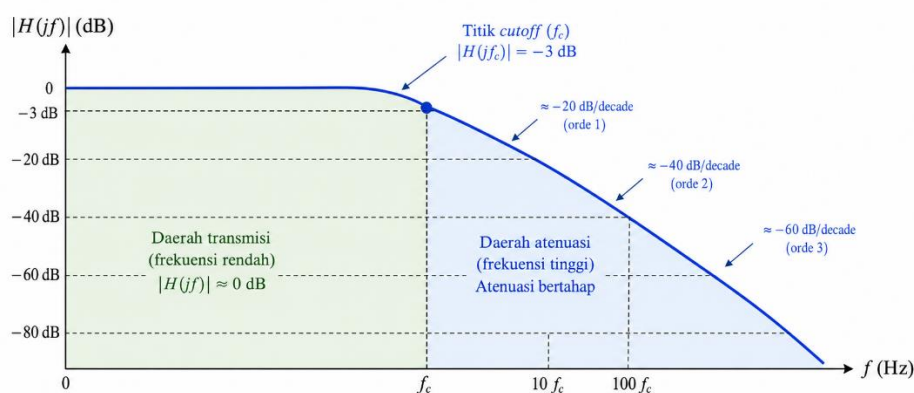
Gambar 2.14: Modifikasi linimasa waktu sinyal audio digital melalui manipulasi matriks linear, mencakup reduksi dimensi (Trimming) dan ekspansi array (Padding).

2.6 PEMFILTERAN DIGITAL (DIGITAL FILTERING)

Sistem pemfilteran digital (*Digital Filters*) secara komputasi merekayasa pelemahan atau pelolosan energi pada pita-pita spektrum frekuensi sasaran. Secara fungsional, taksonomi filter direpresentasikan oleh kurva batas respon frekuensinya.

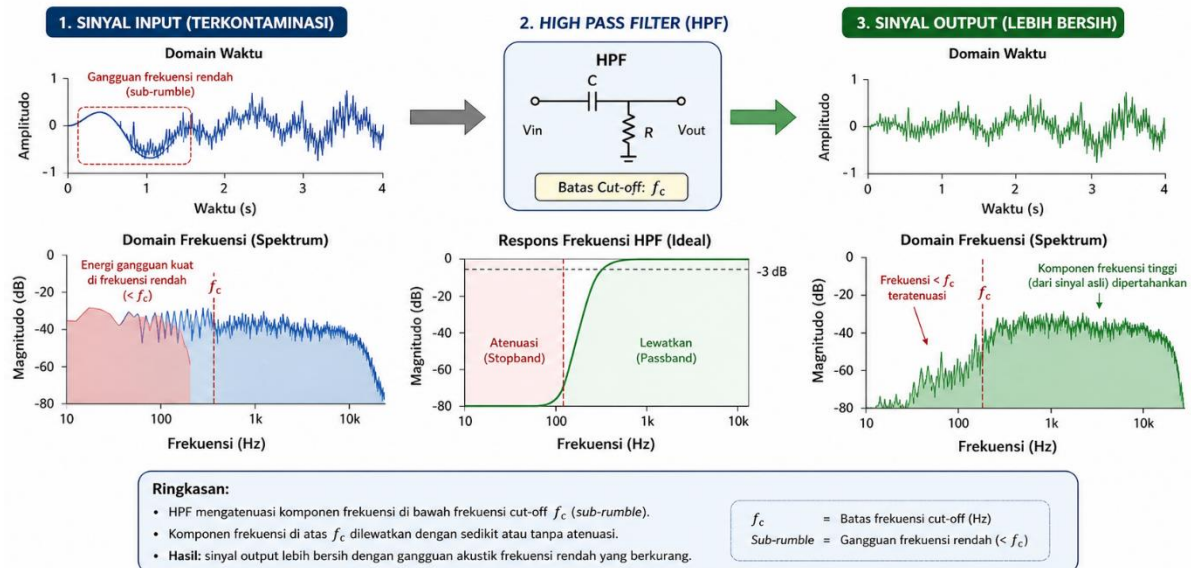
2.6.1 KELAS FILTER KLASIK DASAR

- Low Pass Filter (LPF):** Mereduksi magnitudo sinyal yang melampaui batas frekuensi potong (*Cutoff Frequency* / f_c) dan mentransmisikan nilai frekuensi yang lebih inferior tanpa pelemahan. Mengeliminasi transien intonasi tajam dan merekayasa sentralitas energi pada basis *bass*.



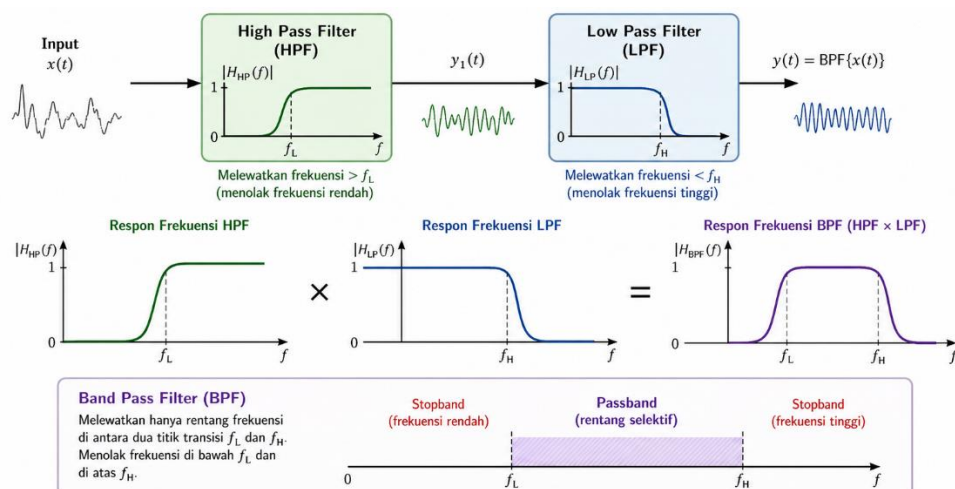
Gambar 2.15: Kurva respon frekuensi Low Pass Filter (LPF) yang menunjukkan transmisi frekuensi rendah di bawah f_c dan atenuasi bertahap pada komponen frekuensi tinggi.

2. **High Pass Filter (HPF):** Bertindak berkebalikan dari LPF, mengatenuasi sinyal bermagnitudo di bawah pita f_c dan membiarkan batas atasnya berlalu bebas. Secara konvensional digunakan untuk pembersihan efek akustik frekuensi rendah tak berguna (seperti *sub-rumble*).



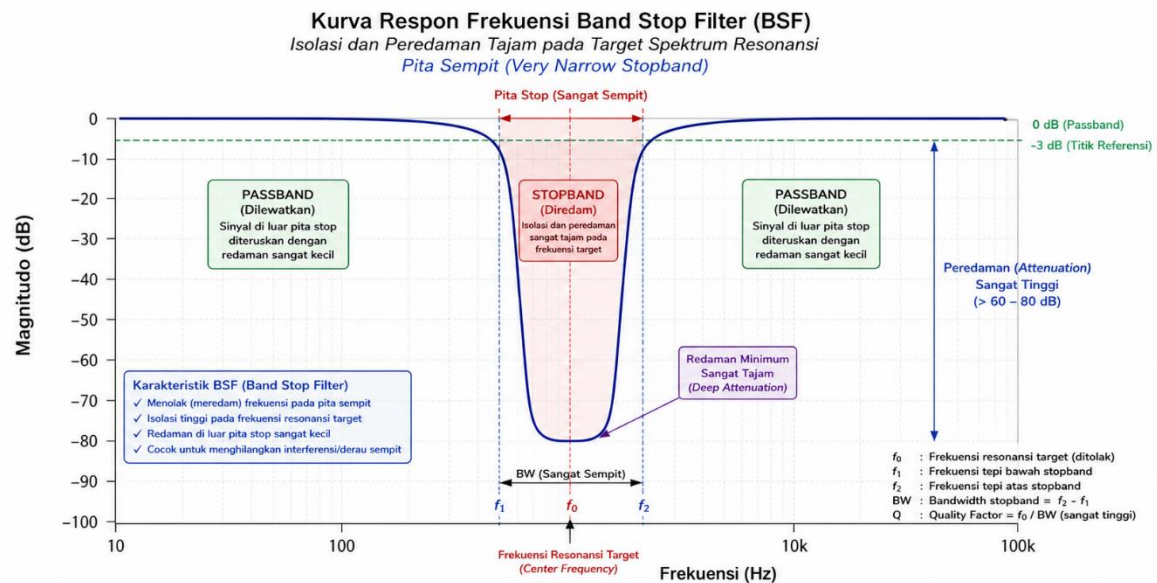
Gambar 2.16: Implementasi High Pass Filter (HPF) untuk mengatenuasi gangguan akustik frekuensi rendah (*sub-rumble*) di bawah batas f_c guna menghasilkan sinyal yang lebih bersih.

3. **Band Pass Filter (BPF):** Mengintegrasikan fungsionalitas HPF dan LPF; mengekstraksi dan menyisakan rentang frekuensi selektif diapit oleh titik transisi Bawah dan Atas, memitigasi pita eksternalnya menjadi diam. Parameter ini mendominasi industri arsitektur telekomunikasi suara.



Gambar 2.17: Mekanisme Band Pass Filter (BPF) sebagai integrasi fungsi HPF dan LPF yang mengekstraksi rentang frekuensi selektif di antara dua titik transisi.

4. **Band Stop Filter (BSF) / Notch Filter:** Kebalikan presisi dari BPF. Mengisolir dan menghancurkan target spektrum resonansi frekuensi berdimensi teramat sempit. Relevan diimplementasikan untuk penindasan dengungan induksi kelistrikan (distorsi AC mains 50 Hz/60 Hz).

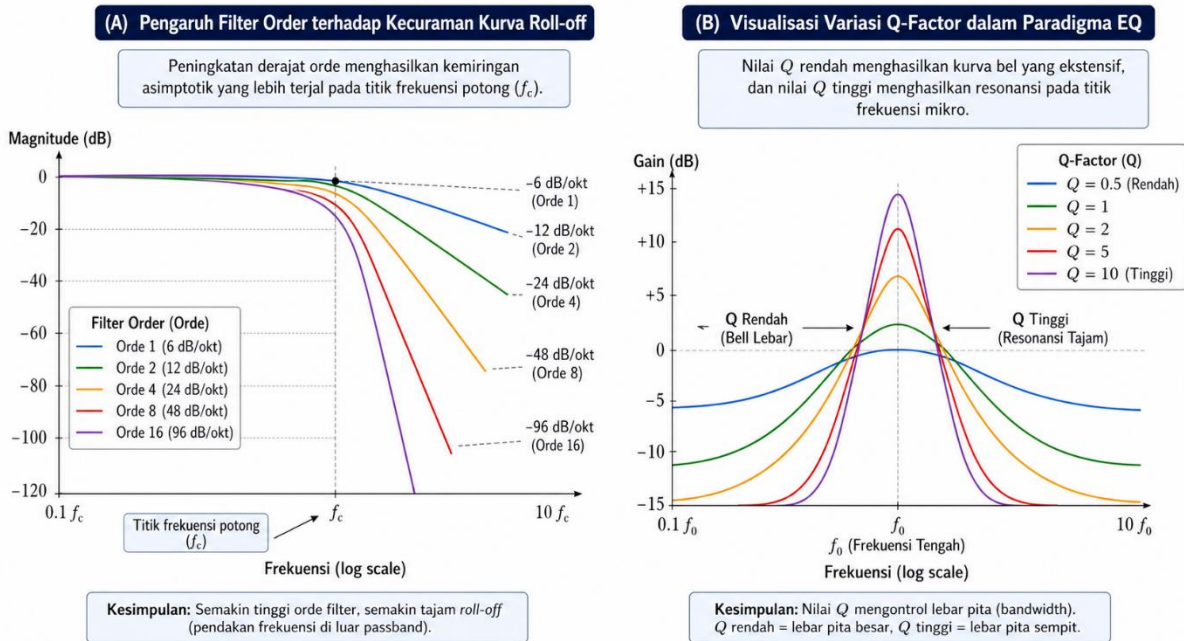


Gambar 2.18: Visualisasi kurva respon frekuensi Band Stop Filter (BSF) yang menunjukkan isolasi dan peredaman tajam pada target spektrum resonansi dengan dimensi pita yang sangat sempit.

2.6.2 KONFIGURASI PARAMETER KUNCI (ORDER DAN Q-FACTOR)

Laju peredaman frekuensi saat melewati perbatasan titik f_c dikendalikan secara mutlak oleh orde komputasi algoritma yang disebut **Orde Filter (Filter Order)**. Peningkatan derajat orde setara dengan peningkatan eksponensial dalam persamaan polinomial yang diproses, yang menerjemahkan kemiringan *roll-off* frekuensi redam menjadi semakin asimptotik secara terjal.

Dalam paradigma kendali penyetara (EQ), kelancipan kurva redam-pancar spesifik diatur dengan menetapkan angka **Faktor Kualitas (Q-Factor)**. Derajat Q marjinal mengakumulasi kurva bel yang ekstensif (melingkupi harmonik berspektrum ekstensif), sementara derajat Q agung mengerucutkan resonansi pengolahan pada frekuensi pin-poin mikroskopik.



Gambar 2.19: Pengaruh Filter Order terhadap kecuraman kurva roll-off; peningkatan derajat orde menghasilkan kemiringan asimptotik yang lebih terjal pada titik frekuensi potong (f_c) dan Visualisasi variasi Q-Factor dalam paradigma EQ, di mana nilai Q rendah menghasilkan kurva bel yang ekstensif dan nilai Q tinggi menghasilkan resonansi pada titik frekuensi mikro.

2.7 PEMROSESAN EFEK SPASIAL DAN LINIMASA WAKTU

2.7.1 GEMA (DELAY) DAN REVERBERATION

- **Delay (Echo):** Di bawah manipulasi matematis, efek gema atau *delay* diproses melalui persamaan penundaan linier konvensional:

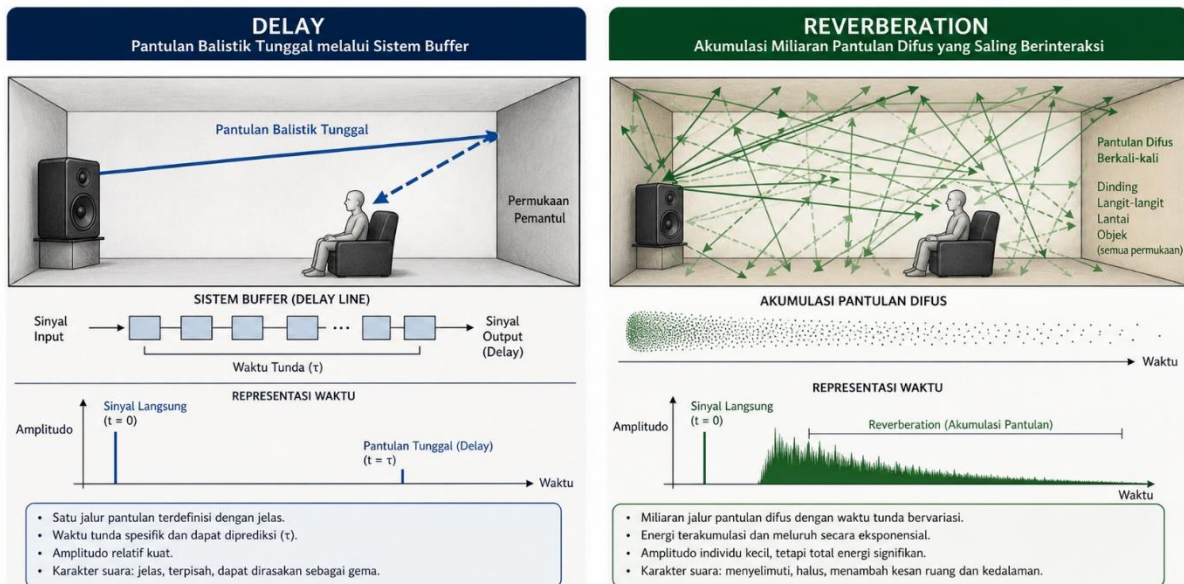
$$y[n] = x[n] + \alpha \cdot x[n - D]$$

Persamaan ini menjumlahkan elemen *buffer* sinyal orisinil ($x[n]$) dengan salinannya yang telah tertunda sejauh jarak sampel D diiringi kompensasi pelemahan redaman (faktor atenuasi berskala α). Model linear abstrak ini secara fisis menirukan pantulan balistik tunggal dari dinding masif tanpa elemen friksi akustik tingkat lanjut.

- **Reverberation (Reverb):** Sebagai model perluasan psikoakustik ruang, *Reverberation* menstimulasikan kerumitan fisis kalkulasi miliaran interaksi serapan gema difus yang memantul dan saling bertabrakan serentak. Pada komputasi modern, pemrosesan ruang direkayasa lewat perhitungan matematis intensif bertajuk Konvolusi (*Convolution*):

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n - k]$$

Di mana deret matriks mentah suara (x) dikalikan-silang terhadap matriks sampel karakteristik respon impuls (*Impulse Response* / h) dari sebuah ruang auditorium nyata untuk mengakuisisi jejak otentik suasananya secara absolut.

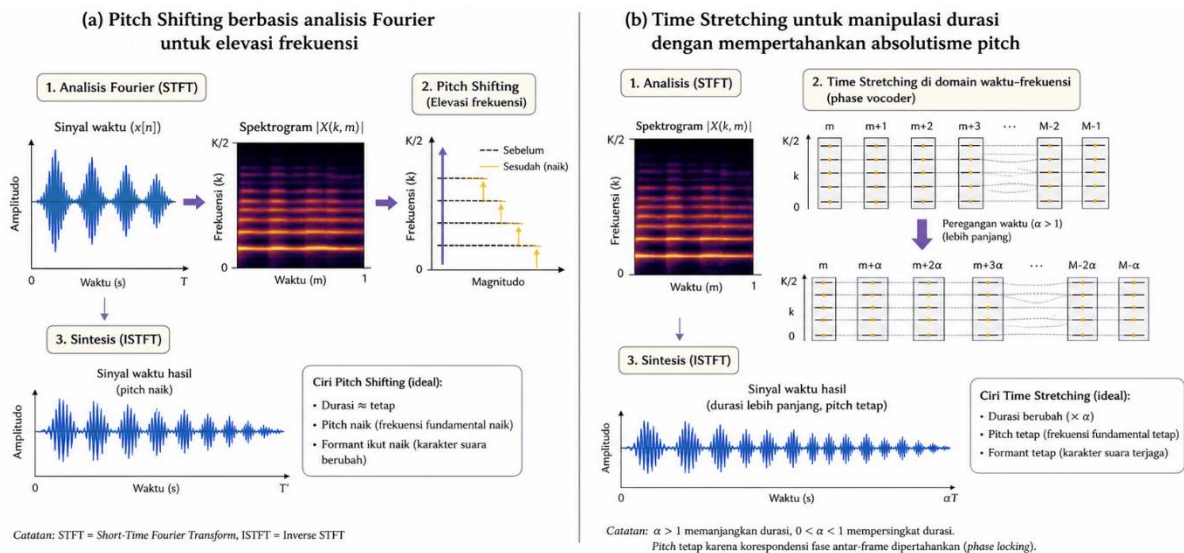


Gambar 2.20: Perbandingan representasi fisis antara Delay sebagai pantulan balistik tunggal melalui sistem buffer dan Reverberation sebagai akumulasi miliaran pantulan difus yang saling berinteraksi.

2.7.2 PERGESERAN PITCH (PITCH SHIFTING) & PEREGANGAN WAKTU (TIME STRETCHING)

- **Pitch Shifting:** Modifikasi komputasional kompleks yang mereplikasi elevasi atau deposisi frekuensi komponen *pitch* tanpa menimbulkan distorsi rasio dimensi waktu putar dari gelombang utama. Dimensi jaraknya terkalibrasi ke dalam satuan derajat notasi *semitone*. Intervensi konvensional dihindari dengan mengkomputasi algoritma berbasis analisis Fourier terbalik seperti *Phase Vocoder*, guna melindungi profil struktur resonansi asli suara manusia (timbre dasar) supaya tidak mengalami pergeseran irelevan.

- **Time Stretching:** Tindakan interpolasi waktu guna mempercepat atau memperlambat rotasi pemutaran rekaman (skala laju) secara utuh, namun mempertahankan absolutisme landasan frekuensi (pitch) dasar secara independen tanpa distorsi. Pendekatan purba dengan mentransformasikan matriks cuplikan secara linear niscaya akan berefek naiknya pitch yang berujung ke insiden defleksi "sindrom suara tupai" tak beraturan.



Gambar 2.21: Teknik modifikasi audio tingkat lanjut: (a) Pitch Shifting berbasis analisis Fourier untuk elevasi frekuensi dan (b) Time Stretching untuk manipulasi durasi dengan mempertahankan absolutisme pitch.

PRAKTIKUM

DEPENDENSI

CODE

```
# =====
# IMPORT LIBRARY (DEPENDENSI)
# =====

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from scipy import signal
from scipy.io import wavfile
from scipy.fft import fft, fftfreq
import librosa
import librosa.display
import soundfile as sf
import IPython.display as ipd
import warnings
import os

warnings.filterwarnings('ignore')

# KONFIGURASI PLOT
plt.rcParams['figure.figsize'] = (14, 4)
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 11
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3
plt.rcParams['axes.spines.top'] = False
plt.rcParams['axes.spines.right'] = False

print("=" * 50)
print("Library berhasil diimport!")
print("=" * 50)
```

CODE

```
# Buat folder output
os.makedirs('audio_output', exist_ok=True)
```

CODE

```
# =====
# FUNGSI HELPER (GUNAKAN DI SELURUH PRAKTIKUM)
# =====

def plot_waveform(audio, sr, title="Waveform Audio",
color='steelblue'):
    """
    Menampilkan visualisasi waveform sinyal audio

    Parameters:
    -----
    audio : np.array → data audio
    sr     : int      → sample rate (Hz)
    title  : str      → judul grafik
    color  : str      → warna grafik
    """
    time = np.linspace(0, len(audio) / sr, num=len(audio))

    plt.figure(figsize=(14, 3))
    plt.plot(time, audio, color=color, linewidth=0.8, alpha=0.9)
    plt.title(f"{title}", fontsize=13, fontweight='bold')
    plt.xlabel("Waktu (detik)")
    plt.ylabel("Amplitudo")
    plt.xlim([0, time[-1]])
    plt.tight_layout()
    plt.show()

def plot_spectrum(audio, sr, title="Spektrum Frekuensi",
color='tomato'):
    """
    Menampilkan spektrum frekuensi sinyal audio menggunakan FFT

    Parameters:
    -----
    audio : np.array → data audio
    sr     : int      → sample rate (Hz)
    title  : str      → judul grafik
    color  : str      → warna grafik
    """
    N = len(audio)
    T = 1.0 / sr
    yf = fft(audio)
    xf = fftfreq(N, T)[:N//2]
    magnitude = 2.0/N * np.abs(yf[:N//2])

    plt.figure(figsize=(14, 3))
    plt.plot(xf, magnitude, color=color, linewidth=0.8)
```

```

plt.title(f"{title}", fontsize=13, fontweight='bold')
plt.xlabel("Frekuensi (Hz)")
plt.ylabel("Magnitude")
plt.xlim([0, sr//2])
plt.tight_layout()
plt.show()

def plot_spectrogram(audio, sr, title="Spectrogram"):
    """
    Menampilkan spectrogram sinyal audio (waktu vs frekuensi)

    Parameters:
    -----
    audio : np.array → data audio
    sr     : int      → sample rate (Hz)
    title  : str      → judul grafik
    """
    plt.figure(figsize=(14, 4))
    D = librosa.amplitude_to_db(
        np.abs(librosa.stft(audio)), ref=np.max
    )
    librosa.display.specshow(
        D, sr=sr, x_axis='time', y_axis='hz', cmap='magma'
    )
    plt.colorbar(format='%+2.0f dB')
    plt.title(f"{title}", fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.show()

def play_audio(audio, sr, label="Audio"):
    """
    Memutar audio langsung di notebook

    Parameters:
    -----
    audio : np.array → data audio
    sr     : int      → sample rate (Hz)
    label  : str      → keterangan audio
    """
    print("=" * 45)
    print(f"\nMemutar: {label}")
    print(f"    - Durasi : {len(audio)/sr:.2f} detik")
    print(f"    - Sample Rate: {sr} Hz")
    print(f"    - Max Amplitude: {np.max(np.abs(audio)):.4f}")
    print()
    return ipd.display(ipd.Audio(audio, rate=sr))

```

```

def save_audio(audio, sr, filename):
    """
    Menyimpan audio ke file WAV

    Parameters:
    -----
    audio      : np.array   → data audio
    sr         : int        → sample rate (Hz)
    filename   : str        → nama file output
    """
    filepath = f"audio_output/{filename}"
    sf.write(filepath, audio, sr)
    print(f"Audio disimpan: {filepath}")

def audio_info(audio, sr, label="Audio"):
    """
    Menampilkan informasi lengkap sinyal audio
    """
    duration = len(audio) / sr
    print(f"\n{'='*45}")
    print(f"      INFO AUDIO: {label}")
    print(f"{'='*45}")
    print(f"  Sample Rate      : {sr:,} Hz")
    print(f"  Jumlah Sampel    : {len(audio):,}")
    print(f"  Durasi           : {duration:.3f} detik")
    print(f"  Max Amplitude    : {np.max(audio):.4f}")
    print(f"  Min Amplitude    : {np.min(audio):.4f}")
    print(f"  RMS Energy       : {np.sqrt(np.mean(audio**2)):.4f}")
    print(f"{'='*45}\n")

print("Semua fungsi helper berhasil didefinisikan!")
print("\nFungsi yang tersedia:")
print("  - plot_waveform()   → visualisasi gelombang")
print("  - plot_spectrum()   → visualisasi spektrum FFT")
print("  - plot_spectrogram() → visualisasi spectrogram")
print("  - play_audio()      → memutar audio")
print("  - save_audio()       → menyimpan audio")
print("  - audio_info()       → informasi audio")

```

3.1 GENERATE SINYAL AUDIO SEDERHANA

3.1.1 GENERATE GELOMBANG SINUS (NADA TUNGGAL)

Kode berikut mendemonstrasikan proses pembuatan, visualisasi, dan pemutaran sinyal sinusoid (*tone generator*). Proses dimulai dengan mendefinisikan parameter utama sinyal seperti *sample rate* sebesar 44100 Hz, durasi 2 detik, amplitudo 0,5, dan frekuensi 440 Hz (merepresentasikan nada A4 atau La). Berdasarkan parameter tersebut, deret waktu diskrit dibentuk untuk kemudian diolah menjadi gelombang sinus menggunakan operasi matematika trigonometri dari pustaka NumPy. Setelah sinyal digital berhasil dibangkitkan, program akan mengeksekusi beberapa fungsi tambahan untuk menampilkan informasi metrik audio, membuat plot visualisasi *waveform* pada 10 milidetik pertama agar pola periodiknya terlihat jelas, serta menggambar grafik spektrum frekuensi. Keseluruhan tahapan diakhiri dengan menyimpan sinyal tersebut ke dalam file audio berekstensi WAV dan memutar suaranya secara langsung.

CODE

```
# =====
# 3.1.1 MEMBUAT SINYAL SINUSOID (TONE GENERATOR)
# =====

# TEORI:
# Gelombang sinusoid adalah sinyal paling dasar dalam audio
# Rumus:  $y(t) = A * \sin(2\pi * f * t)$ 
# dimana:
#   A = Amplitudo (volume)
#   f = Frekuensi (Hz)
#   t = Waktu (detik)
# =====

# --- PARAMETER SINYAL ---
SR          = 44100    # Sample rate standar (Hz)
DURATION    = 2.0      # Durasi sinyal (detik)
AMPLITUDE   = 0.5      # Amplitudo (0.0 - 1.0)
FREQUENCY   = 440.0    # Frekuensi nada A4 (Hz) = nada LA

# --- BUAT ARRAY WAKTU ---
t = np.linspace(0, DURATION, int(SR * DURATION), endpoint=False)

# --- GENERATE SINYAL SINUS ---
tone_440 = AMPLITUDE * np.sin(2 * np.pi * FREQUENCY * t)

# --- TAMPILKAN INFORMASI ---
```

```

audio_info(tone_440, SR, f"Sine Wave {FREQUENCY}Hz")

# --- VISUALISASI WAVEFORM (tampilkan 0.01 detik pertama) ---
time_full = np.linspace(0, DURATION, len(tone_440))
n_show = int(0.01 * SR) # 0.01 detik = 441 sampel

plt.figure(figsize=(14, 3))
plt.plot(time_full[:n_show] * 1000, tone_440[:n_show],
         color='steelblue', linewidth=2)
plt.title(f"Gelombang Sinus - Frekuensi {FREQUENCY} Hz (10ms pertama)",
         fontsize=13, fontweight='bold')
plt.xlabel("Waktu (ms)")
plt.ylabel("Amplitudo")
plt.tight_layout()
plt.show()

# --- VISUALISASI SPEKTRUM ---
plot_spectrum(tone_440, SR, f"Spektrum Frekuensi - Tone {FREQUENCY} Hz")

# --- SIMPAN & PUTAR AUDIO ---
save_audio(tone_440, SR, "01_tone_440hz.wav")
play_audio(tone_440, SR, f"Nada Sinusoid {FREQUENCY} Hz (Nada LA)")

```

3.1.2 GENERATE BERBAGAI JENIS GELOMBANG

Kode berikut mendemonstrasikan proses pembangkitan, visualisasi, dan penyimpanan lima jenis gelombang audio fundamental, yaitu gelombang sinus, kotak, segitiga, gergaji, dan *white noise*. Proses diawali dengan inisialisasi parameter dasar seperti *sample rate* 44100 Hz, durasi 0,5 detik, dan frekuensi 440 Hz untuk menyusun deret waktu. Setiap jenis gelombang kemudian disintesis secara berurutan menggunakan perhitungan matematika dari pustaka NumPy serta modul *scipy.signal*. Setelah seluruh sinyal berhasil dibangkitkan, program memvisualisasikannya dalam bentuk grafik *subplot* vertikal yang menampilkan pola 20 milidetik pertama beserta metrik nilai RMS (*Root Mean Square*) dari masing-masing gelombang. Rangkaian tahapan ini diakhiri dengan pembuatan direktori otomatis untuk mengekspor setiap sinyal menjadi file audio berekstensi WAV secara terpisah, lalu memutarnya satu per satu agar perbedaan karakteristik suara dari tiap jenis gelombang dapat dianalisis secara langsung.

CODE

```
# =====
# 3.1.2 Generate Berbagai Jenis Gelombang
# =====

print("Membuat berbagai jenis gelombang...\n")

# Parameter umum
fs = 44100
duration = 0.5      # 0.5 detik saja agar lebih jelas
f = 440
t = np.linspace(0, duration, int(fs * duration), endpoint=False)

# — 1. Gelombang Sinus (Sine Wave) —————
sine = 0.5 * np.sin(2 * np.pi * f * t)

# — 2. Gelombang Kotak (Square Wave) —————
square = 0.5 * signal.square(2 * np.pi * f * t)

# — 3. Gelombang Segitiga (Triangle Wave) —————
triangle = 0.5 * signal.sawtooth(2 * np.pi * f * t, width=0.5)

# — 4. Gelombang Gergaji (Sawtooth Wave) —————
sawtooth = 0.5 * signal.sawtooth(2 * np.pi * f * t)

# — 5. White Noise —————
np.random.seed(42)  # Untuk reproducibility
white_noise = 0.3 * np.random.randn(len(t))

# — Visualisasi —————
fig, axes = plt.subplots(5, 1, figsize=(14, 18))

waveforms = [
    (sine,      'Sine Wave (440 Hz)',      'steelblue'),
    (square,    'Square Wave (440 Hz)',    'coral'),
    (triangle,  'Triangle Wave (440 Hz)',  'green'),
    (sawtooth,  'Sawtooth Wave (440 Hz)',  'purple'),
    (white_noise, 'White Noise',          'gray')
]

# Hanya tampilkan 20ms pertama agar lebih jelas
show_samples = int(0.02 * fs)  # 20ms = 882 sampel

for i, (wave, title, color) in enumerate(waveforms):
    axes[i].plot(t[:show_samples] * 1000, wave[:show_samples],
                 color=color, linewidth=1.5)
    axes[i].set_title(title, fontweight='bold')
    axes[i].set_ylabel('Amplitudo')
```

```

axes[i].set_ylim(-0.8, 0.8)
axes[i].grid(True, alpha=0.3)

# Info statistik
rms = np.sqrt(np.mean(wave**2))
axes[i].text(0.98, 0.85, f'RMS: {rms:.4f}',
             transform=axes[i].transAxes,
             ha='right', fontsize=10,
             bbox=dict(boxstyle='round', facecolor='wheat',
alpha=0.5))

axes[-1].set_xlabel('Waktu (ms)')
plt.suptitle('Jenis-jenis Gelombang Audio', fontsize=16,
fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

# Simpan file audio untuk setiap gelombang
import os
os.makedirs('audio_files', exist_ok=True)

for wave, title, _ in waveforms:
    filename = f"audio_files/{title.replace(' ', '_').lower()}.wav"
    sf.write(filename, wave, fs)

print("\nSemua file audio tersimpan di folder audio_files/")
print("\nCoba putar masing-masing gelombang dan perhatikan perbedaan suaranya!")

# Putar semua gelombang
print("\n" + "="*50)
print("    PUTAR BERBAGAI JENIS GELOMBANG")
print("="*50)

for wave, title, _ in waveforms:
    play_audio(wave, fs, title)

```

3.1.3 PERBANDINGAN BERBAGAI FREKUENSI (TANGGA NADA)

Pembuatan tangga nada musik dilakukan dengan mendefinisikan sebuah kamus frekuensi dari C4 (261.63 Hz) hingga C5 (523.25 Hz) serta mengatur parameter dasar audio yang krusial, yaitu *sampling rate* sebesar 44100 Hz, durasi 0.5 detik per nada, dan amplitudo 0.4. Proses dilanjutkan dengan melakukan iterasi pada setiap nilai frekuensi untuk menghasilkan gelombang sinus dasar, di mana sebuah teknik penerapan *envelope* (amplop linier) ditambahkan pada 10 milidetik pertama dan terakhir (*fade-in* dan *fade-out*) guna menghaluskan transisi dan menghindari artefak letupan suara (*audio click*) saat pergantian nada. Setiap gelombang nada tunggal tersebut kemudian digabungkan secara sekuensial menggunakan fungsi konkatenasi *array* untuk membentuk satu melodi utuh, sementara 20 milidetik pertama dari masing-masing nada diplot secara individual ke dalam kisi *subplot* 4x2 guna memvisualisasikan perbedaan kerapatan periode gelombang akibat variasi frekuensi. Pada tahap akhir, sinyal gabungan yang merepresentasikan keseluruhan tangga nada tersebut diekspor menjadi berkas audio berekstensi WAV dan diputar untuk verifikasi pendengaran.

CODE

```
# =====
# 3.1.3 TANGGA NADA - PERBANDINGAN BERBAGAI FREKUENSI
# =====
#
# TEORI:
# Setiap nada musik memiliki frekuensi yang berbeda:
#   Do = 261.63 Hz (C4)
#   Re = 293.66 Hz (D4)
#   Mi = 329.63 Hz (E4)
#   Fa = 349.23 Hz (F4)
#   Sol= 392.00 Hz (G4)
#   La = 440.00 Hz (A4)
#   Si = 493.88 Hz (B4)
# =====

# --- DEFINISI TANGGA NADA ---
notes = {
    'Do (C4)' : 261.63,
    'Re (D4)' : 293.66,
    'Mi (E4)' : 329.63,
    'Fa (F4)' : 349.23,
    'Sol (G4)' : 392.00,
    'La (A4)' : 440.00,
    'Si (B4)' : 493.88,
    'Do (C5)' : 523.25,
}
```

```

SR          = 44100
DURATION    = 0.5    # 0.5 detik per nada
AMP         = 0.4

# --- VISUALISASI PERBANDINGAN ---
fig, axes = plt.subplots(4, 2, figsize=(14, 12))
axes = axes.flatten()

audio_sequence = np.array([]) # Gabungkan semua nada

for idx, (name, freq) in enumerate(notes.items()):
    t = np.linspace(0, DURATION, int(SR * DURATION), endpoint=False)

    # Apply envelope (fade in/out) agar tidak klik
    tone = AMP * np.sin(2 * np.pi * freq * t)
    env = np.ones_like(tone)
    fade = int(0.01 * SR)
    env[:fade] = np.linspace(0, 1, fade) # fade in
    env[-fade:] = np.linspace(1, 0, fade) # fade out
    tone = tone * env

    # Gabungkan jadi melodi
    audio_sequence = np.concatenate([audio_sequence, tone])

    # Plot waveform (20ms pertama)
    n_show = int(0.02 * SR)
    time_ms = np.linspace(0, 20, n_show)

    axes[idx].plot(time_ms, tone[:n_show], color='steelblue',
linewidth=1.5)
    axes[idx].set_title(f"{name} = {freq} Hz", fontweight='bold')
    axes[idx].set_xlabel("Waktu (ms)")
    axes[idx].set_ylabel("Amplitudo")
    axes[idx].set_ylim([-0.5, 0.5])

plt.suptitle("Perbandingan Gelombang Tangga Nada Do-Re-Mi",
            fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

# --- SIMPAN & PUTAR MELODI ---
save_audio(audio_sequence, SR, "02_tangga_nada.wav")
play_audio(audio_sequence, SR, "Melodi Tangga Nada Do-Re-Mi-Fa-Sol-
La-Si-Do")

```

3.1.4 GELOMBANG KOMPLEKS (GABUNGAN SINUSOID)

Pembuatan sinyal kompleks direalisasikan berdasarkan prinsip Teorema Fourier melalui penggabungan beberapa gelombang sinusoid untuk membentuk sebuah *chord* C Mayor. Proses diawali dengan menetapkan parameter *sampling rate* sebesar 44100 Hz dan durasi waktu 2.0 detik, kemudian membangkitkan tiga komponen gelombang dasar: nada fundamental Do/C4 (261.63 Hz, amplitudo 0.4), harmonik pertama Mi/E4 (329.63 Hz, amplitudo 0.3), dan harmonik kedua Sol/G4 (392.00 Hz, amplitudo 0.3). Ketiga komponen sinyal individual tersebut dijumlahkan secara linear menjadi satu *array* gabungan, yang selanjutnya melalui tahap normalisasi dengan cara membagi sinyal terhadap nilai maksimum absolutnya dan diskalakan ke faktor 0.8 guna mencegah terjadinya distorsi atau *clipping* pada amplitudo audio. Untuk keperluan analisis, bentuk gelombang dari ketiga komponen tunggal beserta hasil gabungannya divisualisasikan pada rentang waktu 10 milidetik pertama, diikuti dengan pemanggilan fungsi analisis spektrum domain frekuensi untuk memvalidasi kemunculan tiga puncak frekuensi diskrit pembentuk *chord*, sebelum akhirnya sinyal komposit tersebut diekspor menjadi berkas audio berekstensi WAV dan diinstruksikan untuk diputar.

CODE

```
# =====
# 3.1.4 SINYAL KOMPLEKS - GABUNGAN BEBERAPA SINUSOID
# =====
#
# TEORI:
# Teorema Fourier menyatakan bahwa sinyal kompleks apapun
# dapat dipecah menjadi jumlahan sinusoid dengan frekuensi
# dan amplitudo yang berbeda (harmonik).
#
# Contoh: Chord musik = beberapa nada dimainkan bersamaan
# =====

SR          = 44100
DURATION    = 2.0
t           = np.linspace(0, DURATION, int(SR * DURATION),
endpoint=False)

# --- BUAT KOMPONEN FREKUENSI ---
f1, a1 = 261.63, 0.4 # Do (C4) - fundamental
f2, a2 = 329.63, 0.3 # Mi (E4) - harmonik 1
f3, a3 = 392.00, 0.3 # Sol (G4) - harmonik 2

component1 = a1 * np.sin(2 * np.pi * f1 * t)
component2 = a2 * np.sin(2 * np.pi * f2 * t)
component3 = a3 * np.sin(2 * np.pi * f3 * t)

# --- GABUNGAN (CHORD C MAYOR) ---
```

```

chord_C = component1 + component2 + component3
chord_C = chord_C / np.max(np.abs(chord_C)) * 0.8 # Normalisasi

# --- VISUALISASI ---
fig, axes = plt.subplots(4, 1, figsize=(14, 12))
n_show = int(0.01 * SR)
time_ms = np.linspace(0, 10, n_show)

colors = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0']
signals = [component1, component2, component3, chord_C]
titles = [
    f"Komponen 1: Do (C4) = {f1} Hz, Amplitudo={a1}",
    f"Komponen 2: Mi (E4) = {f2} Hz, Amplitudo={a2}",
    f"Komponen 3: Sol (G4) = {f3} Hz, Amplitudo={a3}",
    "Gabungan Chord C Mayor (Do + Mi + Sol)"
]

for ax, sig, title, color in zip(axes, signals, titles, colors):
    ax.plot(time_ms, sig[:n_show], color=color, linewidth=2)
    ax.set_title(f"{title}", fontweight='bold')
    ax.set_xlabel("Waktu (ms)")
    ax.set_ylabel("Amplitudo")

plt.suptitle("Pembentukan Chord dari Gabungan Sinusoid",
             fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- SPEKTRUM CHORD ---
plot_spectrum(chord_C, SR, "Spektrum Chord C Mayor (Terlihat 3 Puncak Frekuensi)")

# --- SIMPAN & PUTAR ---
save_audio(chord_C, SR, "03_chord_C_mayor.wav")
play_audio(chord_C, SR, "Chord C Mayor (Do + Mi + Sol)")

```

3.1.5 PENGARUH SAMPLING RATE

Demonstrasi pengaruh *sampling rate* terhadap kualitas sinyal direalisasikan dengan membangkitkan sebuah sinyal uji gelombang sinus berfrekuensi fundamental 1000 Hz yang dibandingkan melintasi empat variasi resolusi frekuensi *sampling* diskrit: 4000 Hz, 8000 Hz, 16000 Hz, dan standar audio 44100 Hz. Proses visualisasi dilakukan dengan memetakan sinyal referensi analog (garis putus-putus) terhadap representasi sinyal digitalnya menggunakan metode *step plot* dan *scatter plot* pada jendela waktu 5 milidetik pertama guna memperlihatkan secara eksplisit perbedaan kerapatan titik sampel. Aspek teoritis fundamental yang diimplementasikan dalam skrip ini adalah evaluasi otomatis terhadap Teorema Nyquist, di mana program mengkalkulasi batas frekuensi Nyquist ($f_s/2$) untuk setiap skenario guna memvalidasi kemampuan rekonstruksi sinyal dan mendeteksi potensi terjadinya fenomena *aliasing* secara matematis. Sebagai tahap konfirmasi empiris, keseluruhan sinyal pada berbagai *sampling rate* tersebut diregenerasi dengan durasi 1 detik, diekspor menjadi berkas berekstensi WAV, dan diputar secara sekuensial agar kompresi atau degradasi kualitas audio dapat dikomparasi secara langsung.

CODE

```
# =====
# 3.1.5 Demonstrasi Pengaruh Sampling Rate
# =====

# Sinyal asli dengan sampling rate tinggi (referensi)
fs_original = 44100
duration = 0.5
f_test = 1000 # 1 kHz
t_original = np.linspace(0, duration, int(fs_original * duration))
signal_original = np.sin(2 * np.pi * f_test * t_original)

# Berbagai sampling rate untuk dibandingkan
sampling_rates = [4000, 8000, 16000, 44100]
labels = ['4,000 Hz\n(Kualitas buruk)', '8,000 Hz\n(Telepon)',
          '16,000 Hz\n(Wideband)', '44,100 Hz\n(CD Quality)']
colors = ['red', 'orange', 'gold', 'green']

fig, axes = plt.subplots(len(sampling_rates), 1, figsize=(14, 16))

show_duration = 0.005 # Tampilkan 5ms
t_show_max = 0.005

for i, (fs_test, label, color) in enumerate(zip(sampling_rates,
labels, colors)):
    t_test = np.linspace(0, duration, int(fs_test * duration))
    signal_test = np.sin(2 * np.pi * f_test * t_test)
```

```

# Filter sampel untuk ditampilkan
mask = t_test <= t_show_max

# Plot
axes[i].step(t_test[mask] * 1000, signal_test[mask],
             color=color, linewidth=2, where='mid', label='Sinyal
Digital')
axes[i].plot(t_original[t_original <= t_show_max] * 1000,
            signal_original[t_original <= t_show_max],
            'k--', linewidth=1, alpha=0.5, label='Sinyal Asli')

# Tandai titik sampel
axes[i].scatter(t_test[mask] * 1000, signal_test[mask],
               color=color, s=50, zorder=5)

n_samples_shown = np.sum(mask)
axes[i].set_title(f'fs = {label} | {n_samples_shown} sampel dalam
5ms',
                 fontweight='bold', color=color)
axes[i].set_ylabel('Amplitudo')
axes[i].set_ylim(-1.5, 1.5)
axes[i].grid(True, alpha=0.3)
axes[i].legend(loc='upper right')

# Hitung Nyquist
f_nyquist = fs_test / 2
nyquist_ok = "(V) OK" if f_nyquist > f_test else "(X) ALIASING!"
axes[i].text(0.02, 0.1, f'Nyquist: {f_nyquist} Hz {nyquist_ok}',
            transform=axes[i].transAxes, fontsize=9,
            bbox=dict(boxstyle='round',
facecolor='lightyellow'))

axes[-1].set_xlabel('Waktu (ms)')
plt.suptitle(f'Pengaruh Sampling Rate terhadap Sinyal {f_test} Hz',
            fontsize=16, fontweight='bold', y=1)
plt.tight_layout(h_pad=1.5)
plt.show()

print("\n KESIMPULAN PERCOBAAN 1.4:")
print(f" Sinyal uji: {f_test} Hz")
for fs_test, label in zip(sampling_rates, labels):
    nyquist = fs_test / 2
    status = "(V) Memenuhi Nyquist" if nyquist > f_test else "(X)
ALIASING TERJADI!"
    print(f" fs={fs_test:6d} Hz | Nyquist={nyquist:6.0f} Hz |
{status}")

```

```

print("\n" + "="*50)
print("    DENGARKAN PENGARUH SAMPLING RATE")
print("="*50)

# Gunakan durasi yang lebih panjang (1 detik) agar lebih jelas
terdengar
duration_audio = 1.0

for fs_test, label in zip(sampling_rates, labels):
    t_audio = np.linspace(0, duration_audio, int(fs_test *
duration_audio))
    signal_audio = np.sin(2 * np.pi * f_test * t_audio)

    filename = f"audio_files/fs_{fs_test}hz.wav"
    sf.write(filename, signal_audio, fs_test)
    play_audio(signal_audio, fs_test, f"Sampling Rate {fs_test} Hz")

```

3.1.6 PENGARUH BIT DEPTH

Demonstrasi pengaruh *bit depth* terhadap kualitas sinyal direalisasikan melalui simulasi proses kuantisasi amplitudo menggunakan sebuah fungsi khusus yang memetakan rentang nilai sinyal kontinu ke dalam level diskrit sebanyak $2^{\text{bit_depth}}$. Proses diawali dengan mendefinisikan sinyal referensi sinusoidal berfrekuensi 440 Hz pada *sampling rate* 44100 Hz, yang kemudian direduksi resolusi amplitudonya pada empat skenario kedalaman bit: 3-bit, 8-bit, 16-bit, dan 24-bit. Aspek teoritis krusial yang diimplementasikan mencakup komputasi *Signal-to-Noise Ratio* (SNR) berdasarkan selisih daya sinyal asli dengan *quantization error*, perhitungan nilai *Root Mean Square* (RMS) dari eror tersebut, serta estimasi *Dynamic Range* yang secara matematis meningkat sekitar 6 dB untuk setiap penambahan 1 bit resolusi. Secara visual, sinyal hasil kuantisasi dipetakan menggunakan *step plot* bersama grid garis level diskrit pada jendela waktu 10 milidetik guna mengilustrasikan secara eksplisit distorsi bentuk tangga (*staircase effect*) akibat pembulatan amplitudo, sebelum sinyal tersebut diregenerasi dengan durasi 1 detik dan diekspor ke format WAV agar kemunculan derau kuantisasi (*quantization noise*) dapat dianalisis secara komparatif melalui persepsi auditori.

CODE

```
# =====
# 3.1.6 Demonstrasi Pengaruh Bit Depth (Quantization)
# =====

def quantize(signal_data, bit_depth):
    """
    Fungsi kuantisasi sinyal ke bit depth tertentu

    Parameter:
        signal_data : array sinyal (-1 hingga 1)
        bit_depth   : kedalaman bit (mis: 8, 16, 24)

    Return:
        Sinyal yang sudah dikuantisasi
    """
    # Jumlah level kuantisasi
    levels = 2 ** bit_depth

    # Normalisasi ke range [0, levels]
    signal_norm = (signal_data + 1) * (levels / 2)

    # Bulatkan ke integer terdekat
    signal_quantized = np.round(signal_norm)

    # Clip ke range yang valid
    signal_quantized = np.clip(signal_quantized, 0, levels - 1)

    # Kembalikan ke range [-1, 1]
    signal_output = (signal_quantized / (levels / 2)) - 1

    return signal_output

# --- HITUNG SNR (Signal to Noise Ratio) ---
def calculate_snr(clean, noisy):
    noise = noisy - clean
    snr = 10 * np.log10(np.mean(clean**2) / np.mean(noise**2))
    return snr

# Sinyal referensi (resolusi sangat tinggi = "analog")
fs = 44100
duration = 0.01 # 10ms untuk zoom
t = np.linspace(0, duration, int(fs * duration))
signal_ref = np.sin(2 * np.pi * 440 * t)

# Bit depth yang akan dibandingkan
bit_depths = [3, 8, 16, 24]
colors = ['red', 'orange', 'steelblue', 'green']
```

```

fig, axes = plt.subplots(len(bit_depths) + 1, 1, figsize=(14, 20))

# Plot sinyal referensi
axes[0].plot(t * 1000, signal_ref, 'k-', linewidth=2)
axes[0].set_title('Sinyal Referensi (Resolusi Sangat Tinggi)',
fontweight='bold')
axes[0].set_ylabel('Amplitudo')
axes[0].grid(True, alpha=0.3)
axes[0].set_ylim(-1.5, 1.5)

for i, (bits, color) in enumerate(zip(bit_depths, colors)):
    # Kuantisasi
    sig_quantized = quantize(signal_ref, bits)

    # Hitung quantization error
    q_error = sig_quantized - signal_ref
    q_rms = np.sqrt(np.mean(q_error**2))

    # Hitung SNR
    snr = calculate_snr(signal_ref, sig_quantized)

    # Plot
    levels = 2 ** bits
    axes[i+1].step(t * 1000, sig_quantized, color=color,
                    linewidth=2, where='mid')
    axes[i+1].plot(t * 1000, signal_ref, 'k--', linewidth=1,
                    alpha=0.4, label='Referensi')

    # Gambar garis horizontal untuk setiap level kuantisasi
    q_levels = np.linspace(-1, 1, min(levels, 20))
    for ql in q_levels:
        axes[i+1].axhline(y=ql, color='gray', linewidth=0.3,
alpha=0.3)

    axes[i+1].set_title(
        f'{bits}-bit | {levels:,} level | Dynamic Range: {bits*6:.0f}
dB | SNR: {snr:.1f} dB',
        fontweight='bold', color=color
    )
    axes[i+1].set_ylabel('Amplitudo')
    axes[i+1].set_ylim(-1.5, 1.5)
    axes[i+1].grid(True, alpha=0.3)
    axes[i+1].legend(loc='upper right')

    print(f" {bits:2d}-bit: {levels:10,} level | SNR = {snr:6.1f} dB
| "
        f"Quantization Error RMS = {q_rms:.6f}")

```

```

axes[-1].set_xlabel('Waktu (ms)')
plt.suptitle('Pengaruh Bit Depth terhadap Kualitas Sinyal',
             fontsize=16, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

print("\n" + "="*50)
print("    DENGARKAN PENGARUH BIT DEPTH")
print("="*50)

# Gunakan sinyal yang lebih panjang (1 detik) untuk mendengarkan
perbedaannya
duration_audio = 1.0
t_audio = np.linspace(0, duration_audio, int(fs * duration_audio))
signal_audio_ref = np.sin(2 * np.pi * 440 * t_audio)

for bits in bit_depths:
    sig_quantized = quantize(signal_audio_ref, bits)

    filename = f"audio_files/bitdepth_{bits}bit.wav"
    sf.write(filename, sig_quantized, fs)
    play_audio(sig_quantized, fs, f"Bit Depth {bits}-bit")

```

3.2 MEMBACA FILE AUDIO

3.2.1 LOAD AUDIO DENGAN LIBROSA

Proses analisis audio nyata diawali dengan memuat sampel suara terompet bawaan pustaka `librosa`, di mana penetapan parameter `sr=None` diinstruksikan secara eksplisit untuk mempertahankan nilai *sampling rate* asli tanpa memicu proses *resampling* otomatis, sebelum panjang *array* sinyal dipotong menjadi 5 detik pertama guna mengoptimalkan beban komputasi. Representasi sinyal tersebut kemudian diekstraksi dan divisualisasikan secara komprehensif ke dalam tiga metrik utama menggunakan modul kisi tata letak: plot *waveform* murni untuk melihat fluktuasi amplitudo pada domain waktu, komputasi spektrum *Fast Fourier Transform* (FFT) diskrit yang dipotong hingga batas frekuensi Nyquist untuk mengidentifikasi magnitudo puncak setiap komponen harmonik pada domain frekuensi, serta pembentukan *spectrogram* termal melalui kalkulasi matriks *Short-Time Fourier Transform* (STFT) yang amplitudonya dikonversi ke resolusi logaritmik desibel (dB) guna memetakan persebaran energi frekuensi yang berubah seiring berjalannya waktu. Alur skrip diakhiri dengan pemanggilan fungsi untuk

mendengarkan langsung data *array* sinyal tersebut sebagai bentuk validasi auditori terhadap analisis visual yang telah dikonstruksi.

CODE

```
# =====
# 3.2.1 MEMBACA FILE AUDIO NYATA
# =====
#
# Kita akan menggunakan sample audio dari librosa
# (bisa diganti dengan file audio Anda sendiri)
# =====

# --- OPSI 1: Gunakan sample audio bawaan librosa ---
print("Mengunduh sample audio dari librosa...")
AUDIO_PATH, SR_ORIGINAL = librosa.load(
    librosa.ex('trumpet'), # sample audio terompet
    sr=None                # sr=None = pertahankan sample rate asli
)

# Ambil 5 detik pertama saja
MAX_DURATION = 5 # detik
AUDIO_PATH = AUDIO_PATH[:int(SR_ORIGINAL * MAX_DURATION)]

# --- OPSI 2: Load file audio sendiri (uncomment jika punya file) ---
# AUDIO_PATH, SR_ORIGINAL = librosa.load('nama_file.wav', sr=None)

print("Audio berhasil dimuat!")
audio_info(AUDIO_PATH, SR_ORIGINAL, "Sample Audio (Trumpet)")

# --- VISUALISASI LENGKAP ---
fig = plt.figure(figsize=(14, 10))
gs = gridspec.GridSpec(3, 1, hspace=0.4)

# Plot 1: Waveform
ax1 = fig.add_subplot(gs[0])
time_arr = np.linspace(0, len(AUDIO_PATH)/SR_ORIGINAL,
len(AUDIO_PATH))
ax1.plot(time_arr, AUDIO_PATH, color='steelblue', linewidth=0.5,
alpha=0.9)
ax1.set_title("Waveform Audio Asli", fontsize=12, fontweight='bold')
ax1.set_xlabel("Waktu (detik)")
ax1.set_ylabel("Amplitudo")

# Plot 2: Spektrum FFT
ax2 = fig.add_subplot(gs[1])
N = len(AUDIO_PATH)
yf = fft(AUDIO_PATH)
```

```

xf = fftfreq(N, 1/SR_ORIGINAL)[:N//2]
mag = 2.0/N * np.abs(yf[:N//2])
ax2.plot(xf, mag, color='tomato', linewidth=0.7)
ax2.set_title("Spektrum Frekuensi (FFT)", fontsize=12,
fontweight='bold')
ax2.set_xlabel("Frekuensi (Hz)")
ax2.set_ylabel("Magnititude")
ax2.set_xlim([0, SR_ORIGINAL//2])

# Plot 3: Spectrogram
ax3 = fig.add_subplot(gs[2])
D = librosa.amplitude_to_db(np.abs(librosa.stft(AUDIO_PATH)),
ref=np.max)
img = librosa.display.specshow(D, sr=SR_ORIGINAL, x_axis='time',
                                y_axis='hz', cmap='magma', ax=ax3)
fig.colorbar(img, ax=ax3, format='%+2.0f dB')
ax3.set_title("Spectrogram (Waktu vs Frekuensi)", fontsize=12,
fontweight='bold')

plt.suptitle("Analisis Lengkap Audio - Sample Trumpet",
            fontsize=15, fontweight='bold')
plt.show()

# --- PUTAR AUDIO ASLI ---
play_audio(AUDIO_PATH, SR_ORIGINAL, "Audio Asli (Trumpet Sample)")

```

3.3 MANIPULASI DASAR AUDIO

3.3.1 MENGUBAH VOLUME (AMPLITUDO)

Manipulasi volume audio direalisasikan melalui proses penskalaan amplitudo dengan mengalikan *array* sinyal kontinu asli terhadap sebuah faktor pengali linier atau *gain*. Proses diawali dengan mendefinisikan beberapa parameter *gain* mulai dari 0.25 hingga 2.00, yang secara matematis ekuivalen dengan konversi tingkat logaritmik desibel (dB) berdasarkan persamaan $20 \log_{10}(\text{gain})$. Dalam tahap visualisasi, iterasi dilakukan untuk mengaplikasikan setiap nilai *gain* pada sinyal, disertai dengan implementasi algoritma pendeteksi *clipping* yang memvalidasi apakah terdapat nilai absolut amplitudo yang melampaui ambang batas digital 1.0. Gelombang hasil modifikasi ini kemudian diplot pada domain waktu bersama garis referensi pembatas di amplitudo 1.0 dan -1.0 untuk memperlihatkan secara eksplisit terjadinya distorsi akibat amplifikasi berlebih. Pada langkah terakhir, fungsi `np.clip` dieksekusi secara preventif pada sinyal keluaran guna membatasi seluruh nilai amplitudo agar tetap berada dalam rentang aman [-1.0, 1.0] demi mencegah distorsi *hardware*, sebelum

sinyal diekspor menjadi berkas audio berekstensi WAV dan diinisialisasi untuk pemutaran.

CODE

```
# =====
# 3.3.1 MANIPULASI VOLUME (AMPLITUDO SCALING)
# =====
#
# TEORI:
# Volume audio dikontrol dengan mengalikan amplitudo sinyal
# dengan faktor skala (gain):
#   audio_baru = audio_asli * gain
#
# Dalam skala dB:
#   gain_dB = 20 * log10(gain_linear)
#   +6 dB  ≈ volume 2x
#   -6 dB  ≈ volume 0.5x
# =====

# --- GUNAKAN AUDIO DARI CELL 7 ---
audio_asli = AUDIO_PATH.copy()
sr          = SR_ORIGINAL

# --- BUAT VARIASI VOLUME ---
gain_configs = {
    'Volume 25% (Sangat Pelan)' : 0.25,
    'Volume 50% (Pelan)'         : 0.50,
    'Volume Asli (100%)'         : 1.00,
    'Volume 150% (Keras)'        : 1.50,
    'Volume 200% (Sangat Keras)' : 2.00,
}

# --- VISUALISASI PERBANDINGAN ---
fig, axes = plt.subplots(len(gain_configs), 1, figsize=(14, 14))
time_arr  = np.linspace(0, len(audio_asli)/sr, len(audio_asli))

colors_vol = ['#00BCD4', '#2196F3', '#4CAF50', '#FF9800', '#F44336']

for idx, (label, gain) in enumerate(gain_configs.items()):
    audio_modified = audio_asli * gain

    # Clipping indicator
    clipped = np.any(np.abs(audio_modified) > 1.0)
    clip_txt = " (!) CLIPPING!" if clipped else " (V) OK"

    axes[idx].plot(time_arr, audio_modified,
                    color=colors_vol[idx], linewidth=0.5, alpha=0.9)
```

```

axes[idx].axhline(y=1.0, color='red', linestyle='--',
                  linewidth=1, alpha=0.5, label='Batas Clipping')
axes[idx].axhline(y=-1.0, color='red', linestyle='--',
                  linewidth=1, alpha=0.5)
axes[idx].set_title(
    f"{label} | Gain={gain}x | "
    f"dB={20*np.log10(gain):.1f} dB{clip_txt}",
    fontweight='bold'
)
axes[idx].set_ylabel("Amplitudo")
axes[idx].set_ylim([-2.2, 2.2])
axes[idx].legend(loc='upper right', fontsize=8)

axes[-1].set_xlabel("Waktu (detik)")
plt.suptitle("Perbandingan Berbagai Level Volume",
             fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- PUTAR SETIAP VARIASI VOLUME ---
print("\n" + "="*50)
print("    PUTAR PERBANDINGAN VOLUME")
print("="*50)

for label, gain in gain_configs.items():
    audio_vol = audio_asli * gain
    # Clip agar tidak rusak speaker
    audio_vol = np.clip(audio_vol, -1.0, 1.0)
    save_audio(audio_vol, sr, f"04_volume_{int(gain*100)}pct.wav")
    play_audio(audio_vol, sr, label)

```

3.3.2 NORMALISASI AUDIO

Implementasi teknik normalisasi audio direalisasikan untuk menyesuaikan dan mengoptimalkan level amplitudo sinyal tanpa mendistorsi karakteristik fundamentalnya, diawali dengan mensimulasikan hasil rekaman berintensitas rendah melalui pengalihan *array* sinyal asli dengan faktor reduksi 0.1. Proses penskalaan sinyal ini dikonstruksi menggunakan dua metode matematis yang secara mendasar berbeda: *Peak Normalization* yang bekerja dengan mengevaluasi nilai amplitudo absolut tertinggi dari sinyal untuk kemudian dikalikan dengan rasio pengali agar mencapai ambang batas puncak yang diinginkan (melalui penetapan parameter `target_peak=0.9`), serta *RMS Normalization* yang melakukan proses penskalaan berdasarkan kalkulasi akar kuadrat rata-rata energi sinyal secara keseluruhan menuju target level energi yang ditetapkan (menggunakan parameter `target_rms=0.1`). Perbedaan karakteristik amplifikasi dari kedua algoritma normalisasi tersebut

selanjutnya divalidasi secara komprehensif melalui perbandingan visual *subplot* domain waktu dengan garis batas referensi 0.9, yang turut mengkalkulasi dan memetakan perubahan metrik *Peak* dan RMS secara numerik, sebelum sinyal keluaran diekspor menjadi berkas audio digital berformat WAV dan dieksekusi pemutarannya guna memfasilitasi komparasi persepsi auditori.

CODE

```
# =====
# 3.3.2 NORMALISASI AUDIO
# =====
#
# TEORI:
# Normalisasi = menyesuaikan amplitudo audio ke nilai standar
# tanpa mengubah karakter sinyal.
#
# Jenis Normalisasi:
# 1. Peak Normalization : max amplitude = 1.0
# 2. RMS Normalization  : average energy = target
# 3. Loudness Norm      : menggunakan standar LUFS (broadcasting)
# =====

audio_asli = AUDIO_PATH.copy()
sr          = SR_ORIGINAL

# Buat audio yang sangat pelan (simulasi recording lemah)
audio_lemah = audio_asli * 0.1

print(f"Audio Lemah   - Max Amplitude:
{np.max(np.abs(audio_lemah)):.4f}")
print(f"Audio Lemah   - RMS Energy    :
{np.sqrt(np.mean(audio_lemah**2)):.4f}")

# --- METODE 1: PEAK NORMALIZATION ---
def peak_normalize(audio, target_peak=1.0):
    """Normalisasi berdasarkan nilai puncak maksimum"""
    peak = np.max(np.abs(audio))
    if peak == 0:
        return audio
    return audio * (target_peak / peak)

# --- METODE 2: RMS NORMALIZATION ---
def rms_normalize(audio, target_rms=0.1):
    """Normalisasi berdasarkan rata-rata energi (RMS)"""
    rms = np.sqrt(np.mean(audio**2))
    if rms == 0:
        return audio
```

```

        return audio * (target_rms / rms)

# --- APLIKASIKAN NORMALISASI ---
audio_peak_norm = peak_normalize(audio_lemah, target_peak=0.9)
audio_rms_norm  = rms_normalize(audio_lemah, target_rms=0.1)

# --- VISUALISASI PERBANDINGAN ---
fig, axes = plt.subplots(3, 1, figsize=(14, 10))
time_arr  = np.linspace(0, len(audio_asli)/sr, len(audio_asli))

configs = [
    (audio_lemah,      'Audio Lemah (Sebelum Normalisasi)',
    '#FF9800'),
    (audio_peak_norm, 'Setelah Peak Normalization (target peak=0.9)',
    '#4CAF50'),
    (audio_rms_norm,  'Setelah RMS Normalization (target RMS=0.1)',
    '#2196F3'),
]

for ax, (audio, title, color) in zip(axes, configs):
    ax.plot(time_arr, audio, color=color, linewidth=0.5, alpha=0.9)
    ax.axhline(y=0.9, color='red', linestyle='--', linewidth=1,
alpha=0.4)
    ax.axhline(y=-0.9, color='red', linestyle='--', linewidth=1,
alpha=0.4)

    rms_val  = np.sqrt(np.mean(audio**2))
    peak_val = np.max(np.abs(audio))

    ax.set_title(
        f"{title}\n"
        f"    Peak={peak_val:.4f} | RMS={rms_val:.4f}",
        fontweight='bold'
    )
    ax.set_ylabel("Amplitudo")
    ax.set_ylim([-1.1, 1.1])

axes[-1].set_xlabel("Waktu (detik)")
plt.suptitle("Perbandingan Metode Normalisasi Audio",
            fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- PUTAR PERBANDINGAN ---
print("\n" + "="*50)
print("    DENGARKAN PERBEDAAN NORMALISASI")
print("="*50)

```

```

save_audio(audio_lemah,      sr, "05a_audio_lemah.wav")
save_audio(audio_peak_norm, sr, "05b_peak_normalized.wav")
save_audio(audio_rms_norm,  sr, "05c_rms_normalized.wav")

play_audio(audio_lemah,      sr, "Audio Lemah (Belum Dinormalisasi)")
play_audio(audio_peak_norm, sr, "Setelah Peak Normalization")
play_audio(audio_rms_norm,  sr, "Setelah RMS Normalization")

```

3.3.3 TRIMMING DAN PADDING AUTO

Manipulasi durasi audio direalisasikan melalui tiga teknik fundamental pada domain waktu: pemotongan manual, penghapusan keheningan otomatis, dan penambahan ruang senyap (*padding*). Proses diawali dengan memotong sinyal secara manual menggunakan metode *array slicing*, di mana rentang batas waktu dikonversi terlebih dahulu menjadi indeks sampel diskrit melalui perkalian dengan nilai *sampling rate*. Selanjutnya, pemotongan otomatis dieksekusi menggunakan fungsi `librosa.effects.trim` yang dikendalikan oleh parameter `top_db=20`, sebuah ambang batas logaritmik yang secara algoritmik mendeteksi dan membuang segmen sinyal yang tingkat amplitudonya berada lebih dari 20 desibel di bawah nilai puncak maksimum (*peak*). Untuk memperpanjang durasi dengan keheningan, teknik *padding* diaplikasikan melalui fungsi konkatenasi *array* (`np.concatenate`) guna menyisipkan deretan nilai nol (`np.zeros`)—yang merepresentasikan amplitudo titik senyap—pada durasi waktu spesifik di bagian awal dan akhir data sinyal utama. Perubahan dimensi panjang gelombang dari ketiga modifikasi struktural ini kemudian divalidasi secara visual melalui grafik plot domain waktu sebelum masing-masing varian diekspor menjadi berkas audio digital untuk dianalisis melalui pemutaran langsung.

CODE

```

# =====
# 3.3.3 TRIMMING DAN PADDING AUDIO
# =====
#
# TEORI:
# - Trimming : Memotong/menghapus bagian audio
# - Padding  : Menambahkan keheningan (zero samples)
# - Silence Removal : Menghapus bagian yang senyap otomatis
# =====

audio_asli = AUDIO_PATH.copy()
sr          = SR_ORIGINAL

```

```

# --- METODE 1: TRIM MANUAL ---
start_sec = 0.5 # mulai dari detik ke-0.5
end_sec = 3.0 # hingga detik ke-3.0
audio_trimmed = audio_asli[int(start_sec*sr) : int(end_sec*sr)]
print(f"- Trim Manual: {start_sec}s - {end_sec}s")
print(f"    Durasi asli: {len(audio_asli)/sr:.2f}s → Setelah trim: {len(audio_trimmed)/sr:.2f}s")

# --- METODE 2: TRIM OTOMATIS (hapus keheningan) ---
audio_trimmed_auto, _ = librosa.effects.trim(
    audio_asli,
    top_db=20 # threshold: potong bagian < 20dB dari max
)
print(f"\n- Trim Otomatis (threshold 20dB):")
print(f"    Durasi asli: {len(audio_asli)/sr:.2f}s → Setelah trim: {len(audio_trimmed_auto)/sr:.2f}s")

# --- METODE 3: PADDING ---
pad_sebelum = int(0.5 * sr) # 0.5 detik keheningan di awal
pad_sesudah = int(1.0 * sr) # 1.0 detik keheningan di akhir
audio_padded = np.concatenate([
    np.zeros(pad_sebelum), # keheningan awal
    audio_asli, # audio utama
    np.zeros(pad_sesudah) # keheningan akhir
])
print(f"\n- Padding:")
print(f"    Tambah {pad_sebelum/sr:.1f}s diam di awal + {pad_sesudah/sr:.1f}s diam di akhir")
print(f"    Durasi asli: {len(audio_asli)/sr:.2f}s → Setelah padding: {len(audio_padded)/sr:.2f}s")

# --- VISUALISASI ---
fig, axes = plt.subplots(4, 1, figsize=(14, 13))

audio_list = [
    (audio_asli, 'Audio Asli', '#2196F3'),
    (audio_trimmed, f'Trimmed Manual ({start_sec}s - {end_sec}s)', '#4CAF50'),
    (audio_trimmed_auto, 'Trimmed Otomatis (Hapus Keheningan)', '#FF9800'),
    (audio_padded, f'Padded (+{pad_sebelum/sr:.1f}s awal, +{pad_sesudah/sr:.1f}s akhir)', '#9C27B0'),
]

for ax, (audio, title, color) in zip(axes, audio_list):
    t = np.linspace(0, len(audio)/sr, len(audio))
    ax.plot(t, audio, color=color, linewidth=0.5, alpha=0.9)
    ax.set_title(

```

```

        f"{title} | Durasi: {len(audio)/sr:.2f} detik",
        fontweight='bold'
    )
    ax.set_ylabel("Amplitudo")
    ax.set_xlim([0, len(audio)/sr])

axes[-1].set_xlabel("Waktu (detik)")
plt.suptitle("Teknik Trimming dan Padding Audio",
            fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- SIMPAN & PUTAR ---
save_audio(audio_asli, sr, "06a_asli.wav")
save_audio(audio_trimmed, sr, "06b_trimmed_manual.wav")
save_audio(audio_trimmed_auto, sr, "06c_trimmed_auto.wav")
save_audio(audio_padded, sr, "06d_padded.wav")

print("\n" + "="*50)
print("    PUTAR PERBANDINGAN TRIMMING & PADDING")
print("="*50)
play_audio(audio_asli, sr, "Audio Asli")
play_audio(audio_trimmed, sr, "Audio Setelah Trim Manual")
play_audio(audio_trimmed_auto, sr, "Audio Setelah Trim Otomatis")
play_audio(audio_padded, sr, "Audio Setelah Padding")

```

3.4 DIGITAL FILTERING

3.4.1 LOW PASS FILTER (LPF)

Implementasi *Low Pass Filter* (LPF) akan direalisasikan untuk memblokir komponen frekuensi tinggi dan meneruskan frekuensi rendah dengan mengonstruksi sebuah sinyal komposit yang menggabungkan gelombang fundamental 200 Hz (merepresentasikan *bass*) dan 4000 Hz (merepresentasikan *treble*) pada *sampling rate* 44100 Hz. Pemrosesan sinyal digital ini akan dibangun menggunakan desain filter Butterworth, di mana parameter krusial frekuensi *cutoff* terlebih dahulu dinormalisasi terhadap batas frekuensi Nyquist ($f_s/2$) untuk menentukan titik awal redaman, dikombinasikan dengan penetapan orde filter bernilai 5 guna mengatur tingkat ketajaman kurva transisi atenuasi (*roll-off*). Eksekusi penyaringan gelombang akan mengaplikasikan metode *forward-backward* berfase nol (*zero-phase filtering*) sehingga peluruhan frekuensi tidak akan menyebabkan terjadinya distorsi pergeseran waktu pada sinyal keluaran. Dampak atenuasi spesifik dari filter ini selanjutnya akan diproyeksikan secara komprehensif dengan memvariasikan nilai *cutoff* melintasi

rentang 500 Hz hingga 3000 Hz, yang mana karakteristik penekanannya terhadap gelombang *treble* akan divalidasi silang melalui komparasi visual antara plot *waveform* berdurasi 10 milidetik pada domain waktu dan spektrum magnitudo *Fast Fourier Transform* (FFT) pada domain frekuensi , sebelum akhirnya diekspor menjadi berkas audio untuk keperluan inspeksi auditori langsung.

CODE

```
# =====
# 3.4.1 LOW PASS FILTER (LPF)
# =====
#
# TEORI:
# Low Pass Filter (LPF) = meneruskan frekuensi RENDAH,
# memblokir frekuensi TINGGI.
#
# Parameter utama:
#   - cutoff_freq : frekuensi batas (Hz)
#   - order       : orde filter (makin tinggi = makin tajam)
#
# Aplikasi:
#   - Mengurangi noise frekuensi tinggi
#   - Efek "bassy" / "muddy"
#   - Anti-aliasing filter
# =====

# --- BUAT SINYAL CAMPURAN (BASS + TREBLE) ---
SR          = 44100
DURATION    = 2.0
t           = np.linspace(0, DURATION, int(SR * DURATION),
endpoint=False)

freq_bass   = 200    # Hz - suara bass
freq_treble = 4000   # Hz - suara treble

audio_campuran = (
    0.5 * np.sin(2 * np.pi * freq_bass * t) + # Bass
    0.4 * np.sin(2 * np.pi * freq_treble * t) # Treble
)
audio_campuran /= np.max(np.abs(audio_campuran))
audio_campuran *= 0.8

# --- DESAIN LOW PASS FILTER ---
def apply_lpf(audio, sr, cutoff_hz, order=5):
    """
    Menerapkan Low Pass Filter pada audio
```

```

Parameters:
-----

audio      : np.array → data audio
sr         : int      → sample rate
cutoff_hz  : float    → frekuensi cutoff (Hz)
order      : int      → orde filter
"""

nyquist    = sr / 2
cutoff_norm = cutoff_hz / nyquist # normalisasi ke [0,1]

# Desain Butterworth filter
b, a = signal.butter(order, cutoff_norm, btype='low',
analog=False)

# Terapkan filter (forward-backward untuk phase = 0)
audio_filtered = signal.filtfilt(b, a, audio)
return audio_filtered, b, a

# --- TERAPKAN LPF DENGAN BERBAGAI CUTOFF ---
cutoff_freqs = [500, 1000, 2000, 3000]
fig, axes    = plt.subplots(len(cutoff_freqs)+1, 2, figsize=(14, 16))

# Plot sinyal asli
N      = len(audio_campuran)
yf     = fft(audio_campuran)
xf     = fftfreq(N, 1/SR)[:N//2]
mag    = 2.0/N * np.abs(yf[:N//2])

t_plot = np.linspace(0, DURATION, len(audio_campuran))
n_show = int(0.01 * SR)
t_ms   = np.linspace(0, 10, n_show)

axes[0,0].plot(t_ms, audio_campuran[:n_show], color='#607D8B',
linewidth=2)
axes[0,0].set_title("Audio Asli (Bass 200Hz + Treble 4kHz)",
fontweight='bold')
axes[0,0].set_ylabel("Amplitudo")

axes[0,1].plot(xf, mag, color='#607D8B', linewidth=1.5)
axes[0,1].set_title("Spektrum Audio Asli", fontweight='bold')
axes[0,1].set_ylabel("Magnitude")
axes[0,1].set_xlim([0, 6000])

colors_lpf = ['#E91E63', '#FF5722', '#FF9800', '#4CAF50']

for idx, (cutoff, color) in enumerate(zip(cutoff_freqs, colors_lpf),
1):
    audio_lpf, b, a = apply_lpf(audio_campuran, SR, cutoff)

```

```

    # Plot waveform
    axes[idx,0].plot(t_ms, audio_lpf[:n_show], color=color,
linewidth=2)
    axes[idx,0].set_title(f"LPF Cutoff = {cutoff} Hz",
fontweight='bold')
    axes[idx,0].set_ylabel("Amplitudo")

    # Plot spektrum
    yf_lpf = fft(audio_lpf)
    mag_lpf = 2.0/N * np.abs(yf_lpf[:N//2])
    axes[idx,1].plot(xf, mag_lpf, color=color, linewidth=1.5)
    axes[idx,1].axvline(x=cutoff, color='black', linestyle='--',
linewidth=1.5, label=f'Cutoff {cutoff}Hz')
    axes[idx,1].set_title(f"Spektrum Setelah LPF {cutoff} Hz",
fontweight='bold')
    axes[idx,1].set_ylabel("Magnitudo")
    axes[idx,1].set_xlim([0, 6000])
    axes[idx,1].legend(fontsize=9)

for ax in axes[-1]:
    ax.set_xlabel("Waktu (ms)" if ax == axes[-1,0] else "Frekuensi
(Hz)")

plt.suptitle("Low Pass Filter - Perbandingan Berbagai Cutoff
Frequency",
            fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- PUTAR PERBANDINGAN ---
print("\n" + "="*50)
print("    DENGARKAN EFEK LOW PASS FILTER")
print("="*50)

play_audio(audio_campuran, SR, "Audio Asli (Bass + Treble)")

for cutoff in cutoff_freqs:
    audio_lpf, _, _ = apply_lpf(audio_campuran, SR, cutoff)
    save_audio(audio_lpf, SR, f"07_lpf_{cutoff}hz.wav")
    play_audio(audio_lpf, SR, f"LPF Cutoff = {cutoff} Hz")

```

3.4.2 HIGH PASS FILTER (HPF)

Implementasi *High Pass Filter* (HPF) akan direalisasikan untuk meredam komponen frekuensi rendah dan meneruskan komponen frekuensi tinggi pada sinyal audio komposit yang telah dibuat sebelumnya. Proses penyaringan digital ini dikonstruksi menggunakan desain filter IIR Butterworth melalui penetapan parameter `btype='high'`, di mana parameter frekuensi *cutoff* wajib dinormalisasi terlebih dahulu terhadap batas frekuensi Nyquist ($f_s/2$) untuk menentukan letak titik awal redaman, dikombinasikan dengan penetapan orde filter bernilai 5 yang secara teoritis mendikte tingkat ketajaman transisi kurva atenuasi (*roll-off*). Eksekusi pemfilteran kemudian diaplikasikan menggunakan metode *forward-backward* berfase nol (*zero-phase filtering*) melalui fungsi `filtfilt` guna mencegah distorsi pergeseran waktu pada sinyal keluaran. Karakteristik atenuasi dari rancangan filter ini selanjutnya dianalisis secara komprehensif dengan memvariasikan titik frekuensi *cutoff* (200 Hz, 500 Hz, 1000 Hz, dan 2000 Hz) dan memproyeksikannya ke dalam dua mode visualisasi: grafik respons frekuensi (*frequency response*) yang mengkalkulasi magnitudo redaman dalam resolusi logaritmik desibel (dB) beserta identifikasi titik jatuh -3dB menggunakan fungsi `freqz`, serta komparasi plot *waveform* pada domain waktu untuk mengevaluasi perubahan bentuk gelombang secara langsung, sebelum keseluruhan sinyal termodifikasi tersebut diekspor menjadi format audio WAV untuk keperluan inspeksi auditori.

CODE

```
# =====
# 3.4.2 HIGH PASS FILTER (HPF)
# =====
#
# TEORI:
# High Pass Filter (HPF) = meneruskan frekuensi TINGGI,
# memblokir frekuensi RENDAH.
#
# Aplikasi:
#   - Mengurangi rumble/hum frekuensi rendah
#   - Efek "thin" atau "bright"
#   - Telephone effect (menghilangkan bass)
# =====

# --- GUNAKAN AUDIO SINTETIS YANG SAMA ---
audio_test = audio_campuran.copy()

def apply_hpf(audio, sr, cutoff_hz, order=5):
    """Menerapkan High Pass Filter pada audio"""
    nyquist      = sr / 2
    cutoff_norm  = cutoff_hz / nyquist
```

```

    b, a = signal.butter(order, cutoff_norm, btype='high',
analog=False)
    audio_filtered = signal.filtfilt(b, a, audio)
    return audio_filtered, b, a

# --- VISUALISASI FREQUENCY RESPONSE FILTER ---
fig, axes = plt.subplots(2, 2, figsize=(14, 8))
cutoff_hpf_list = [200, 500, 1000, 2000]
colors_hpf      = ['#9C27B0', '#3F51B5', '#009688', '#8BC34A']

for idx, (cutoff, color) in enumerate(zip(cutoff_hpf_list,
colors_hpf)):
    ax = axes[idx//2, idx%2]

    # Hitung frequency response
    nyquist      = SR / 2
    cutoff_norm = cutoff / nyquist
    b, a = signal.butter(5, cutoff_norm, btype='high', analog=False)
    w, h = signal.freqz(b, a, worN=8000, fs=SR)

    # Plot response
    ax.plot(w, 20 * np.log10(np.abs(h)), color=color, linewidth=2.5)
    ax.axvline(x=cutoff, color='red', linestyle='--',
                linewidth=1.5, label=f'Cutoff {cutoff} Hz')
    ax.axhline(y=-3, color='gray', linestyle=':',
                linewidth=1, alpha=0.8, label='-3dB point')
    ax.set_title(f"HPF Frequency Response - Cutoff {cutoff} Hz",
                fontweight='bold')
    ax.set_xlabel("Frekuensi (Hz)")
    ax.set_ylabel("Magnitudo (dB)")
    ax.set_xlim([0, 8000])
    ax.set_ylim([-60, 5])
    ax.legend(fontsize=9)

plt.suptitle("High Pass Filter - Frequency Response Berbagai Cutoff",
            fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- TERAPKAN HPF PADA AUDIO ---
fig, axes = plt.subplots(len(cutoff_hpf_list)+1, 1, figsize=(14, 13))

time_arr = np.linspace(0, len(audio_test)/SR, len(audio_test))

axes[0].plot(time_arr, audio_test, color='#607D8B', linewidth=0.5)
axes[0].set_title("Audio Asli", fontweight='bold')
axes[0].set_ylabel("Amplitudo")

```

```

for idx, (cutoff, color) in enumerate(zip(cutoff_hpf_list,
colors_hpf), 1):
    audio_hpf, _, _ = apply_hpf(audio_test, SR, cutoff)

    axes[idx].plot(time_arr, audio_hpf, color=color, linewidth=0.5)
    axes[idx].set_title(f"HPF Cutoff = {cutoff} Hz",
fontweight='bold')
    axes[idx].set_ylabel("Amplitudo")

axes[-1].set_xlabel("Waktu (detik)")
plt.suptitle("High Pass Filter - Hasil Pada Sinyal Audio",
            fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- PUTAR PERBANDINGAN HPF ---
print("\n" + "="*50)
print("    DENGARKAN EFEK HIGH PASS FILTER")
print("="*50)

play_audio(audio_test, SR, "Audio Asli")

for cutoff in cutoff_hpf_list:
    audio_hpf, _, _ = apply_hpf(audio_test, SR, cutoff)
    save_audio(audio_hpf, SR, f"08_hpf_{cutoff}hz.wav")
    play_audio(audio_hpf, SR, f"HPF Cutoff = {cutoff} Hz")

```

3.4.3 BAND PASS DAN BAND STOP FILTER

Implementasi *Band Pass Filter* (BPF) dan *Band Stop Filter* (BSF) direalisasikan untuk memanipulasi rentang pita frekuensi spesifik pada sebuah sinyal komposit yang dibangun dari superposisi lima gelombang sinusoidal diskrit (200, 500, 1000, 2000, dan 4000 Hz) pada *sampling rate* 44100 Hz. Proses penyaringan dikonstruksi menggunakan desain filter IIR Butterworth orde 4, di mana parameter frekuensi *cutoff* bawah dan atas wajib dinormalisasi terlebih dahulu terhadap batas frekuensi Nyquist serta diamankan menggunakan fungsi `np.clip` agar tetap berada pada rasio yang valid. Selanjutnya, nilai tersebut diumpankan ke pembuat koefisien filter dengan parameter `btype='band'` untuk meneruskan pita sinyal (diuji pada rentang 400–2500 Hz) atau `btype='bandstop'` untuk menekan pita sinyal tertentu layaknya *notch filter* (diuji pada rentang 800–1200 Hz guna menghilangkan komponen 1000 Hz). Eksekusi penyaringan kemudian diaplikasikan melalui fungsi `filtfilt` demi memastikan tidak terjadinya distorsi pergeseran fase pada domain waktu, sementara efektivitas atenuasinya divalidasi secara komprehensif melalui plot *waveform* dan komparasi spektrum magnitudo *Fast Fourier Transform* (FFT) yang

memvisualisasikan hilangnya puncak energi pada frekuensi target, sebelum keseluruhan varian sinyal tersebut diekspor menjadi format audio WAV untuk keperluan observasi auditori secara langsung.

CODE

```
# =====
# 3.4.3 BAND PASS FILTER & BAND STOP (NOTCH) FILTER
# =====
#
# TEORI:
# Band Pass Filter (BPF):
#   - Meneruskan frekuensi dalam rentang tertentu (low_cut ~
high_cut)
#   - Memblokir frekuensi di luar rentang tersebut
#   - Aplikasi: equalizer, telephone channel
#
# Band Stop / Notch Filter:
#   - Memblokir frekuensi dalam rentang tertentu
#   - Meneruskan frekuensi di luar rentang tersebut
#   - Aplikasi: menghilangkan hum 50/60 Hz dari listrik
# =====

# --- BUAT SINYAL MULTI-FREKUENSI ---
SR      = 44100
DURATION = 2.0
t        = np.linspace(0, DURATION, int(SR * DURATION),
endpoint=False)

# Gabungkan 5 frekuensi berbeda
freqs_multi = [200, 500, 1000, 2000, 4000]
audio_multi = sum(0.2 * np.sin(2 * np.pi * f * t) for f in
freqs_multi)
audio_multi /= np.max(np.abs(audio_multi))
audio_multi *= 0.8

print(f"Sinyal multi-frekuensi: {freqs_multi} Hz")

def apply_bpf(audio, sr, low_hz, high_hz, order=4):
    """Menerapkan Band Pass Filter"""
    nyquist = sr / 2
    low_n    = low_hz / nyquist
    high_n   = high_hz / nyquist
    low_n    = np.clip(low_n, 0.001, 0.999)
    high_n   = np.clip(high_n, 0.001, 0.999)
    b, a = signal.butter(order, [low_n, high_n], btype='band',
analog=False)
    return signal.filtfilt(b, a, audio)
```

```

def apply_bsf(audio, sr, low_hz, high_hz, order=4):
    """Menerapkan Band Stop (Notch) Filter"""
    nyquist = sr / 2
    low_n = low_hz / nyquist
    high_n = high_hz / nyquist
    low_n = np.clip(low_n, 0.001, 0.999)
    high_n = np.clip(high_n, 0.001, 0.999)
    b, a = signal.butter(order, [low_n, high_n], btype='bandstop',
analog=False)
    return signal.filtfilt(b, a, audio)

# --- TERAPKAN FILTER ---
# BPF: ambil hanya frekuensi 400Hz - 2500Hz
audio_bpf = apply_bpf(audio_multi, SR, low_hz=400, high_hz=2500)

# BSF: hapus frekuensi 800Hz - 1200Hz (sekitar 1kHz)
audio_bsf = apply_bsf(audio_multi, SR, low_hz=800, high_hz=1200)

# --- VISUALISASI SPEKTRUM ---
fig, axes = plt.subplots(3, 2, figsize=(14, 12))
N = len(audio_multi)
xf = fftfreq(N, 1/SR)[:N//2]

signals_bp = [
    (audio_multi, "Audio Asli (200+500+1k+2k+4k Hz)", '#607D8B',
None, None),
    (audio_bpf, "Band Pass Filter (400-2500 Hz)", '#2196F3',
400, 2500),
    (audio_bsf, "Band Stop Filter (800-1200 Hz)", '#F44336',
800, 1200),
]

n_show = int(0.015 * SR)
t_ms = np.linspace(0, 15, n_show)

for idx, (audio, title, color, low, high) in enumerate(signals_bp):
    yf = fft(audio)
    mag = 2.0/N * np.abs(yf[:N//2])

    # Waveform
    axes[idx,0].plot(t_ms, audio[:n_show], color=color, linewidth=2)
    axes[idx,0].set_title(f"{title}", fontweight='bold')
    axes[idx,0].set_ylabel("Amplitudo")
    axes[idx,0].set_xlabel("Waktu (ms)")

    # Spektrum
    axes[idx,1].plot(xf, mag, color=color, linewidth=1.5)

```

```

        if low and high:
            axes[idx,1].axvspan(low, high, alpha=0.15,
                                color='green' if 'Pass' in title else
'red',
                                label=f'{'Pass' if "Pass" in title else
"Stop"} Region')

        # Mark frekuensi asli
        for f in freqs_multi:
            axes[idx,1].axvline(x=f, color='orange', linestyle=':',
                                linewidth=1.2, alpha=0.7)

        axes[idx,1].set_title(f"Spektrum - {title}", fontweight='bold')
        axes[idx,1].set_ylabel("Magnitude")
        axes[idx,1].set_xlabel("Frekuensi (Hz)")
        axes[idx,1].set_xlim([0, 6000])
        if low and high:
            axes[idx,1].legend(fontsize=9)

plt.suptitle("Band Pass Filter vs Band Stop Filter",
             fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- PUTAR PERBANDINGAN ---
print("\n" + "="*50)
print("    DENGARKAN EFEK BAND FILTER")
print("="*50)

save_audio(audio_multi, SR, "09a_multi_freq.wav")
save_audio(audio_bpf, SR, "09b_band_pass.wav")
save_audio(audio_bsf, SR, "09c_band_stop.wav")

play_audio(audio_multi, SR, "Audio Asli (5 Frekuensi Campuran)")
play_audio(audio_bpf, SR, "Setelah Band Pass Filter (400-2500 Hz)")
play_audio(audio_bsf, SR, "Setelah Band Stop Filter (800-1200 Hz)")

```

3.5 EFEK AUDIO

3.5.1 NOISE REDUCTION

Pembuatan sistem simulasi penambahan dan reduksi derau (*noise*) audio diawali dengan membangkitkan sinyal referensi murni berupa gelombang sinusoidal berfrekuensi 440 Hz. Sinyal tersebut kemudian didistorsi menggunakan dua karakteristik derau sintetik: *white noise* yang memiliki energi merata di seluruh pita frekuensi yang dibangkitkan melalui fungsi distribusi normal `np.random.randn` pada parameter level amplitudo 0.15, serta *pink noise* yang dikonstruksi dengan memproses derau putih melewati filter IIR (`signal.lfilter`) guna mensimulasikan peluruhan energi 3 dB per oktaf. Untuk memulihkan kualitas sinyal, dua teknik *denoising* diimplementasikan, yaitu *Low Pass Filter* (LPF) Butterworth orde 6 dengan titik *cutoff* dinormalisasi pada 800 Hz untuk memblokir interferensi frekuensi tinggi, serta *Median Filter* non-linier menggunakan ukuran *kernel* 7 (`scipy.ndimage.median_filter`) untuk meratakan anomali amplitudo pada domain waktu secara langsung. Keberhasilan redaman derau tersebut dievaluasi secara objektif melalui kalkulasi matematis *Signal-to-Noise Ratio* (SNR) menggunakan persamaan $10 \log_{10}$, yang membandingkan rata-rata daya sinyal murni terhadap daya derau sisa, sebelum divalidasi lebih lanjut melalui komparasi grafik *waveform* dan pemutaran berkas audio keluaran berformat WAV.

CODE

```
# =====
# 3.5.1 PENAMBAHAN DAN PENGURANGAN NOISE
# =====
#
# TEORI:
# Noise = sinyal yang tidak diinginkan yang mengganggu audio
#
# Jenis Noise:
# 1. White Noise : energi merata di semua frekuensi (suara "shhhh")
# 2. Pink Noise : energi menurun 3dB per oktaf (lebih natural)
# 3. Gaussian Noise: distribusi normal (Gaussian)
#
# Teknik Pengurangan Noise Sederhana:
# 1. Spectral Subtraction
# 2. Low Pass Filtering
# 3. Median Filtering
# =====
#
# --- BUAT SINYAL BERSIH ---
SR = 44100
```

```

DURATION = 2.0
t          = np.linspace(0, DURATION, int(SR * DURATION),
endpoint=False)
audio_clean = 0.6 * np.sin(2 * np.pi * 440 * t)

# --- TAMBAHKAN BERBAGAI JENIS NOISE ---
np.random.seed(42)

# White Noise
noise_level = 0.15
white_noise = noise_level * np.random.randn(len(t))
audio_noisy_white = audio_clean + white_noise

# Pink Noise (aproksimasi)
def generate_pink_noise(n_samples):
    white = np.random.randn(n_samples)
    b = [0.049922035, -0.095993537, 0.050612699, -0.004408786]
    a = [1, -2.494956002, 2.017265875, -0.522189400]
    pink = signal.lfilter(b, a, white)
    return pink / np.max(np.abs(pink))

pink_noise = noise_level * generate_pink_noise(len(t))
audio_noisy_pink = audio_clean + pink_noise

# --- DENOISE MENGGUNAKAN LPF ---
def simple_denoise_lpf(audio_noisy, sr, cutoff=1000, order=6):
    """Denoise sederhana menggunakan Low Pass Filter"""
    nyquist = sr / 2
    cutoff_norm = cutoff / nyquist
    b, a = signal.butter(order, cutoff_norm, btype='low')
    return signal.filtfilt(b, a, audio_noisy)

# --- DENOISE MENGGUNAKAN MEDIAN FILTER ---
def simple_denoise_median(audio_noisy, kernel_size=5):
    """Denoise menggunakan Median Filter"""
    from scipy.ndimage import median_filter
    return median_filter(audio_noisy, size=kernel_size)

audio_denoised_lpf = simple_denoise_lpf(audio_noisy_white, SR,
cutoff=800)
audio_denoised_median = simple_denoise_median(audio_noisy_white,
kernel_size=7)

# --- HITUNG SNR (Signal to Noise Ratio) ---
def calculate_snr(clean, noisy):
    noise = noisy - clean
    snr = 10 * np.log10(np.mean(clean**2) / np.mean(noise**2))
    return snr

```

```

snr_noisy    = calculate_snr(audio_clean, audio_noisy_white)
snr_lpf      = calculate_snr(audio_clean, audio_denoised_lpf)
snr_median   = calculate_snr(audio_clean, audio_denoised_median)

print(f"SNR Sebelum Denoise (White Noise): {snr_noisy:.2f} dB")
print(f"SNR Setelah LPF Denoise           : {snr_lpf:.2f} dB")
print(f"SNR Setelah Median Denoise       : {snr_median:.2f} dB")
print(f"\nSNR makin tinggi = kualitas makin baik")

# --- VISUALISASI ---
fig, axes = plt.subplots(5, 1, figsize=(14, 18))
time_arr = np.linspace(0, DURATION, len(audio_clean))

plot_data = [
    (audio_clean,          "Audio Bersih (Sinyal",
    440Hz)",              '#2196F3', None),
    (audio_noisy_white,    f"+ White Noise (level={noise_level})",
    SNR={snr_noisy:.1f}dB", '#F44336', None),
    (audio_noisy_pink,     f"+ Pink Noise",
    (level={noise_level})), '#FF9800', None),
    (audio_denoised_lpf,   f"LPF Denoise (cutoff=800Hz)",
    SNR={snr_lpf:.1f}dB",   '#4CAF50', None),
    (audio_denoised_median, f"Median Denoise (kernel=7)",
    SNR={snr_median:.1f}dB", '#9C27B0', None),
]

for ax, (audio, title, color, _) in zip(axes, plot_data):
    ax.plot(time_arr, audio, color=color, linewidth=0.6, alpha=0.9)
    ax.set_title(f"{title}", fontweight='bold')
    ax.set_ylabel("Amplitudo")

axes[-1].set_xlabel("Waktu (detik)")
plt.suptitle("Noise Reduction Pada Sinyal Audio",
             fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- PUTAR PERBANDINGAN ---
print("\n" + "="*50)
print("    DENGARKAN PERBEDAAN NOISE REDUCTION")
print("="*50)

save_audio(audio_clean,          SR, "10a_clean.wav")
save_audio(audio_noisy_white,    SR, "10b_noisy_white.wav")
save_audio(audio_noisy_pink,     SR, "10c_noisy_pink.wav")
save_audio(audio_denoised_lpf,   SR, "10d_denoised_lpf.wav")
save_audio(audio_denoised_median, SR, "10e_denoised_median.wav")

```

```

play_audio(audio_clean, SR, "Audio Bersih (Referensi)")
play_audio(audio_noisy_white, SR, "Audio + White Noise")
play_audio(audio_noisy_pink, SR, "Audio + Pink Noise")
play_audio(audio_denoised_lpf, SR, "Setelah LPF Denoise")
play_audio(audio_denoised_median, SR, "Setelah Median Denoise")

```

3.5.2 EFEK ECHO DAN REVERB

Implementasi efek *echo* dan *reverb* pada sinyal audio direalisasikan menggunakan dua pendekatan manipulasi domain waktu yang berbeda. Efek *echo* dikonstruksi secara manual melalui fungsi iteratif yang menjumlahkan sinyal asli dengan salinan dirinya sendiri yang digeser (*shifted*) seiring waktu, di mana parameter `delay_sec` menentukan jeda pengulangan, `n_echoes` menentukan jumlah iterasi pantulan, dan decay mengatur peluruhan eksponensial amplitudo pada setiap pantulannya (menggunakan persamaan $y[n] = x[n] + \alpha x[n - \text{delay}]$). Sementara itu, simulasi *reverb* atau gema ruangan diimplementasikan menggunakan teknik konvolusi FFT cepat (`signal.fftconvolve`) antara sinyal audio asli dan sebuah Respon Impuls (*Impulse Response/IR*) sintesis, di mana IR tersebut dibangkitkan dari *array* derau acak (sebagai representasi difusi akustik kompleks) yang diluruhkan amplitudonya secara eksponensial menggunakan fungsi `np.exp` berdasarkan parameter `room_size` (durasi pantulan) dan `damping` (tingkat penyerapan material). Kedua fungsi efek tersebut memuat mekanisme normalisasi mandiri pasca-pemrosesan guna mencegah distorsi *clipping*, sebelum hasilnya diaplikasikan pada sampel terompet dengan berbagai variasi konfigurasi (pendek/panjang, kecil/besar), divisualisasikan fluktuasi *waveform*-nya (di mana terlihat sinyal memanjang atau menebal akibat ekor *delay*), dan diekspor ke format WAV untuk divalidasi kualitas pantulan akustiknya secara auditori.

CODE

```

# =====
# 3.5.2 EFEK ECHO DAN REVERB
# =====
#
# TEORI:
# Echo/Delay    : Pengulangan sinyal setelah jeda waktu tertentu
# Reverb        : Banyak echo dengan delay dan decay berbeda-beda
#                 (simulasi akustik ruangan)
#
# Rumus Echo:
#    $y[n] = x[n] + \alpha * x[n - \text{delay}]$ 
#   dimana:

```

```

#     alpha = decay factor (0 < alpha < 1)
#     delay = waktu tunda dalam sampel
#
# Rumus Reverb (Multi-tap Echo):
#     y[n] = x[n] + Σ alpha_k * x[n - delay_k]
# =====

audio_input = AUDIO_PATH.copy()
sr          = SR_ORIGINAL

# --- EFEK ECHO SEDERHANA ---
def apply_echo(audio, sr, delay_sec=0.3, decay=0.5, n_echoes=3):
    """
    Menerapkan efek echo pada audio

    Parameters:
    -----
    audio      : np.array → data audio
    sr         : int       → sample rate
    delay_sec  : float     → jeda waktu antar echo (detik)
    decay      : float     → faktor peluruhan (0-1)
    n_echoes   : int       → jumlah pengulangan
    """
    delay_samples = int(delay_sec * sr)
    output = audio.copy().astype(np.float64)

    for i in range(1, n_echoes + 1):
        delay_i = delay_samples * i
        decay_i = decay ** i

        if delay_i < len(audio):
            padded = np.zeros(len(audio))
            padded[delay_i:] = audio[:-delay_i] if delay_i > 0 else
audio
            output += decay_i * padded

    # Normalisasi agar tidak clipping
    output = output / np.max(np.abs(output)) * 0.9
    return output

# --- EFEK REVERB (CONVOLUTION REVERB SEDERHANA) ---
def apply_reverb(audio, sr, room_size=0.5, damping=0.5):
    """
    Simulasi reverb ruangan menggunakan multi-tap delay

    Parameters:
    -----
    room_size : float → ukuran ruangan (0-1)

```

```

damping    : float → penyerapan suara dinding (0-1)
"""

# Buat Impulse Response (IR) sintetis
ir_duration = room_size * 2.0 # detik
ir_samples  = int(ir_duration * sr)

# IR: decay eksponensial + random untuk difusi
t_ir = np.linspace(0, ir_duration, ir_samples)
ir    = np.random.randn(ir_samples)
ir *= np.exp(-damping * 5 * t_ir)
ir[0] = 1.0 # direct sound
ir /= np.max(np.abs(ir))

# Konvolusi audio dengan IR
output = signal.fftconvolve(audio, ir, mode='full')[:len(audio)]
output /= np.max(np.abs(output)) * 1.1
return output

# --- TERAPKAN EFEK ---
# Echo dengan delay berbeda
audio_echo_short = apply_echo(audio_input, sr, delay_sec=0.1,
                                decay=0.6, n_echoes=4)
audio_echo_long  = apply_echo(audio_input, sr, delay_sec=0.4,
                                decay=0.5, n_echoes=3)

# Reverb dengan ukuran ruangan berbeda
audio_reverb_small = apply_reverb(audio_input, sr, room_size=0.2,
                                    damping=0.8)
audio_reverb_large = apply_reverb(audio_input, sr, room_size=0.8,
                                    damping=0.3)

# --- VISUALISASI ---
fig, axes = plt.subplots(5, 1, figsize=(14, 18))
time_arr = np.linspace(0, len(audio_input)/sr, len(audio_input))

audio_effects = [
    (audio_input, "Audio Asli (Tanpa",
Efek)", "#607D8B"),
    (audio_echo_short, "Echo Pendek (delay=0.1s, decay=0.6)",
"#2196F3"),
    (audio_echo_long, "Echo Panjang (delay=0.4s, decay=0.5)",
"#E91E63"),
    (audio_reverb_small, "Reverb Ruangan Kecil",
(room=0.2)", "#FF9800"),
    (audio_reverb_large, "Reverb Ruangan Besar",
(room=0.8)", "#9C27B0"),
]

```

```

for ax, (audio, title, color) in zip(axes, audio_effects):
    t = np.linspace(0, len(audio)/sr, len(audio))
    ax.plot(t, audio, color=color, linewidth=0.5, alpha=0.9)
    ax.set_title(f"{title}", fontweight='bold')
    ax.set_ylabel("Amplitudo")

axes[-1].set_xlabel("Waktu (detik)")
plt.suptitle("Efek Echo dan Reverb Pada Audio",
            fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- PUTAR PERBANDINGAN ---
print("\n" + "="*50)
print("    DENGARKAN EFEK ECHO DAN REVERB")
print("="*50)

save_audio(audio_input,          sr, "11a_original.wav")
save_audio(audio_echo_short,     sr, "11b_echo_short.wav")
save_audio(audio_echo_long,      sr, "11c_echo_long.wav")
save_audio(audio_reverb_small,   sr, "11d_reverb_small.wav")
save_audio(audio_reverb_large,   sr, "11e_reverb_large.wav")

play_audio(audio_input,          sr, "Audio Asli")
play_audio(audio_echo_short,     sr, "Echo Pendek (delay=0.1s)")
play_audio(audio_echo_long,      sr, "Echo Panjang (delay=0.4s)")
play_audio(audio_reverb_small,   sr, "Reverb Ruangan Kecil")
play_audio(audio_reverb_large,   sr, "Reverb Ruangan Besar")

```

3.5.3 PITCH SHIFTING

Implementasi efek *pitch shifting* direalisasikan untuk memanipulasi frekuensi fundamental suatu sinyal audio tanpa mendistorsi atau mengubah durasi temporalnya. Proses transformasi ini dikonstruksi dengan memanfaatkan fungsi `librosa.effects.pitch_shift` yang dikendalikan oleh parameter kunci `n_steps` dalam metrik *semitone*, di mana pergeseran bernilai positif akan menggeser frekuensi menuju oktaf yang lebih tinggi (+12 *semitone* ekuivalen dengan frekuensi ganda), sementara nilai negatif akan mereduksinya ke oktaf yang lebih rendah (-12 *semitone* ekuivalen dengan frekuensi separuh). Modifikasi tersebut dieksekusi secara iteratif melintasi rentang -12 hingga +12 *semitone*, di mana setiap variasi sinyal keluaran selanjutnya dikonversi ke dalam domain frekuensi melalui komputasi *Fast Fourier Transform* (FFT). Spektrum magnitudo hasil kalkulasi divisualisasikan menggunakan kisi *subplot* untuk memvalidasi secara eksplisit fenomena pergeseran puncak energi harmonik pada sumbu absis (Hz), sebelum

seluruh gelombang termodifikasi tersebut diekspor menjadi berkas audio berformat WAV guna memfasilitasi komparasi persepsi tinggi rendahnya nada secara auditori.

CODE

```
# =====
# 3.5.3 PITCH SHIFTING (UBAH NADA/PITCH)
# =====
#
# TEORI:
# Pitch Shifting = mengubah frekuensi dasar (nada) sinyal
# tanpa mengubah durasi atau kecepatan putar.
#
# Diukur dalam semitone (nada):
#   +12 semitone = naik 1 oktaf (frekuensi x2)
#   -12 semitone = turun 1 oktaf (frekuensi /2)
#   +1 semitone  = naik 1 nada
#
# Berbeda dengan Time Stretching yang mengubah kecepatan.
# =====

audio_input = AUDIO_PATH.copy()
sr          = SR_ORIGINAL

# --- PITCH SHIFTING MENGGUNAKAN LIBROSA ---
semitones_list = [-12, -6, -3, 0, +3, +6, +12]
pitched_audios = {}

print("  Memproses Pitch Shifting...")
for semitone in semitones_list:
    if semitone == 0:
        pitched = audio_input.copy()
    else:
        pitched = librosa.effects.pitch_shift(
            audio_input,
            sr=sr,
            n_steps=semitone
        )
    pitched_audios[semitone] = pitched
    print(f"    * {semitone:+3d} semitone selesai")

# --- VISUALISASI SPEKTRUM PERBANDINGAN ---
fig, axes = plt.subplots(len(semitones_list), 1, figsize=(14, 20))
colors_ps = plt.cm.RdYlGn(np.linspace(0, 1, len(semitones_list)))

for idx, (semitone, color) in enumerate(zip(semitones_list,
colors_ps)):
    audio = pitched_audios[semitone]
```

```

N      = len(audio)
yf     = fft(audio)
xf     = fftfreq(N, 1/sr)[:N//2]
mag    = 2.0/N * np.abs(yf[:N//2])

axes[idx].plot(xf, mag, color=color, linewidth=1.2, alpha=0.9)

oktaf = semitone / 12
label = f"{semitone:+d} semitone ({oktaf:+.2f} oktaf)"
if semitone == 0:
    label += " ← ORIGINAL"

axes[idx].set_title(f"{label}", fontweight='bold', fontsize=10)
axes[idx].set_ylabel("Magnitude", fontsize=8)
axes[idx].set_xlim([0, min(sr//2, 8000)])

axes[-1].set_xlabel("Frekuensi (Hz)")
plt.suptitle("Pitch Shifting - Perbandingan Spektrum",
            fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- PUTAR SEMUA VARIASI PITCH ---
print("\n" + "="*50)
print("    DENGARKAN BERBAGAI VARIASI PITCH")
print("="*50)

for semitone in semitones_list:
    audio = pitched_audios[semitone]
    fname = f"12_pitch_{semitone:+d}_semitone.wav"
    save_audio(audio, sr, fname)

    label = f"Pitch {semitone:+d} Semitone"
    if semitone == 0:
        label += " (Original)"
    elif semitone > 0:
        label += " (Lebih Tinggi/Melengking)"
    else:
        label += " (Lebih Rendah/Besar)"

    play_audio(audio, sr, label)

```

3.5.4 TIME STRETCHING

Implementasi manipulasi durasi audio direalisasikan melalui teknik *time stretching* yang berfungsi mengubah kecepatan putar sinyal tanpa mendistorsi atau menggeser frekuensi fundamentalnya (nada dasar tetap dipertahankan). Eksekusi transformasi ini dikonstruksi menggunakan fungsi `librosa.effects.time_stretch` yang secara algoritmik dikendalikan oleh parameter rasio pengali atau `rate`; di mana nilai `rate` < 1.0 akan mengekspansi sinyal pada domain waktu sehingga pemutaran melambat (misalnya 0.5x memperpanjang durasi hingga dua kali lipat), sedangkan nilai `rate` > 1.0 akan mengompresi sinyal sehingga pemutaran menjadi lebih cepat. Proses modifikasi ini diaplikasikan secara iteratif melintasi berbagai variasi rasio mulai dari 0.5 hingga 2.0, yang mana setiap perubahannya divalidasi secara visual melalui komparasi grafik plot *waveform* guna memperlihatkan secara eksplisit rentang perbedaan dimensi panjang gelombang audio, sebelum seluruh varian sinyal tersebut diekspor menjadi berkas berformat WAV dan diinisialisasi pemutarannya untuk memfasilitasi evaluasi kecepatan secara auditori.

CODE

```
# =====
# 3.5.4 TIME STRETCHING (UBAH KECEPATAN TANPA UBAH PITCH)
# =====
# TEORI:
# Time Stretching = mengubah durasi/kecepatan audio
# tanpa mengubah pitch (nada).
#
# Rate:
#   rate < 1.0 → audio lebih LAMBAT (diperlambat)
#   rate = 1.0 → kecepatan NORMAL
#   rate > 1.0 → audio lebih CEPAT (dipercepat)
#
# Berbeda dengan pitch shifting yang mengubah nada.
# Digunakan dalam: karaoke, podcast, video editing
# =====

audio_input = AUDIO_PATH.copy()
sr          = SR_ORIGINAL

# --- TIME STRETCHING ---
rates = {
    '0.5x (2x Lebih Lambat)' : 0.5,
    '0.75x (1.33x Lebih Lambat)': 0.75,
    '1.0x (Normal)'          : 1.0,
    '1.25x (Lebih Cepat)'     : 1.25,
    '1.5x (1.5x Lebih Cepat)' : 1.5,
```

```

        '2.0x (2x Lebih Cepat)' : 2.0,
    }

print("Memproses Time Stretching...")
stretched_audios = {}

for label, rate in rates.items():
    if rate == 1.0:
        stretched = audio_input.copy()
    else:
        stretched = librosa.effects.time_stretch(audio_input,
rate=rate)

    stretched_audios[label] = (stretched, rate)
    dur = len(stretched) / sr
    print(f"    * rate={rate:.2f} → Durasi: {dur:.2f}s")

# --- VISUALISASI WAVEFORM ---
fig, axes = plt.subplots(len(rates), 1, figsize=(14, 18))
colors_ts = plt.cm.cool(np.linspace(0, 1, len(rates)))

for idx, ((label, (audio, rate)), color) in enumerate(
    zip(stretched_audios.items(), colors_ts)
):
    t = np.linspace(0, len(audio)/sr, len(audio))
    axes[idx].plot(t, audio, color=color, linewidth=0.5, alpha=0.9)
    axes[idx].set_title(
        f"{label} | Durasi: {len(audio)/sr:.2f}s",
        fontweight='bold'
    )
    axes[idx].set_ylabel("Amplitudo")

axes[-1].set_xlabel("Waktu (detik)")
plt.suptitle("Time Stretching - Mengubah Kecepatan Tanpa Ubah Pitch",
            fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- PUTAR PERBANDINGAN ---
print("\n" + "="*50)
print("    DENGARKAN BERBAGAI KECEPATAN PUTAR")
print("="*50)

for idx, (label, (audio, rate)) in
enumerate(stretched_audios.items()):
    fname = f"13_stretch_{str(rate).replace('.', 'p')}x.wav"
    save_audio(audio, sr, fname)
    play_audio(audio, sr, label)

```

3.6 PROYEK MINI

3.6.1 AUDIO MIXER (GABUNG AUDIO)

Pembuatan sistem *audio mixer* dan *synthesizer* sederhana ini dikonstruksi untuk merangkai berbagai instrumen menjadi sebuah komposisi musik utuh dengan tempo 120 BPM, di mana durasi per ketukan (*beat*) ditetapkan bernilai 0.5 detik untuk mengakomodasi birama 4/4. Secara teknis, setiap instrumen disintesis melalui operasi matematis pada sinyal fundamental: *snare drum* dibentuk dari kombinasi *white noise* dan gelombang sinus 250 Hz yang diberi selubung peluruhan eksponensial (`np.exp`), *hi-hat* dirancang menggunakan derau putih yang difilter melalui *High Pass Filter* (HPF) Butterworth berfrekuensi *cutoff* 8000 Hz, sedangkan *bass* dan melodi diekstraksi dari gelombang sinus murni berdasarkan matriks frekuensi nada diskrit. Proses *sequencing* diimplementasikan menggunakan pemetaan larik biner sebagai pola ketukan (seperti `[1, 0, 0, 0, 1, 0, 0, 0]` pada pola *kick*) untuk mengatur penempatan sampel audio pada indeks waktu yang spesifik. Setelah seluruh matriks *track* terisi penuh, eksekusi *mixing* dilakukan dengan cara menjumlahkan (*superposisi*) setiap *array* sinyal yang sebelumnya telah dikalikan dengan bobot penguatan atau volume (*gain*) spesifik, diakhiri dengan proses normalisasi puncak amplitudo komposit pada rasio 0.9 guna mencegah terjadinya fenomena *clipping* digital, sebelum keseluruhan dinamika gelombang divisualisasikan dan diekspor ke format WAV.

CODE

```
# =====
# 3.6.1 SIMPLE AUDIO MIXER
# =====
#
# TEORI:
# Audio Mixing = menggabungkan beberapa sumber audio
# menjadi satu output dengan kontrol volume masing-masing.
#
#  $Mix = \sum (audio\_i * gain\_i)$ 
#
# Aplikasi:
# - Menambahkan background music pada narasi
# - Mixing instrumen musik
# - Menambahkan sound effects
# =====
SR = 44100
```

```

DURATION = 3.0
t          = np.linspace(0, DURATION, int(SR * DURATION),
endpoint=False)

# --- BUAT BEBERAPA TRACK AUDIO ---
print("Membuat track-track audio...")

# Track 1: Bass drum (kick) - pulsa rendah
def make_kick(sr, duration):
    t      = np.linspace(0, duration, int(sr * duration),
endpoint=False)
    freq = 60 * np.exp(-20 * t) # frekuensi turun
    kick = np.sin(2 * np.pi * freq * t) * np.exp(-15 * t)
    return kick

# Track 2: Snare - noise burst
def make_snare(sr, duration):
    t      = np.linspace(0, duration, int(sr * duration),
endpoint=False)
    noise = np.random.randn(len(t)) * np.exp(-25 * t)
    tone  = np.sin(2 * np.pi * 250 * t) * np.exp(-30 * t)
    return 0.5 * noise + 0.5 * tone

# Track 3: Hi-hat - high frequency noise
def make_hihat(sr, duration):
    t      = np.linspace(0, duration, int(sr * duration),
endpoint=False)
    noise = np.random.randn(len(t)) * np.exp(-80 * t)
    # LPF untuk hi-hat
    b, a  = signal.butter(2, 8000/(sr/2), btype='high')
    return signal.lfilter(b, a, noise)

# Track 4: Bass line (nada bass)
def make_bass(sr, duration, freq=80):
    t      = np.linspace(0, duration, int(sr * duration),
endpoint=False)
    bass = 0.6 * np.sin(2 * np.pi * freq * t)
    env  = np.exp(-3 * (t % 0.5))
    return bass * env

# --- BUAT POLA BEAT (4/4 = 4 ketukan per bar) ---
beat_duration = 0.5 # 0.5 detik per ketukan = 120 BPM
total_beats   = int(DURATION / beat_duration)
samples_beat  = int(beat_duration * SR)
total_samples = int(DURATION * SR)

# Inisialisasi track
track_kick    = np.zeros(total_samples)

```

```

track_snare = np.zeros(total_samples)
track_hihat = np.zeros(total_samples)
track_bass = np.zeros(total_samples)
track_melody = np.zeros(total_samples)

# Pola drum (4/4 time)
kick_pattern = [1, 0, 0, 0, 1, 0, 0, 0] # beat 1 dan 3
snare_pattern = [0, 0, 1, 0, 0, 0, 1, 0] # beat 2 dan 4
hihat_pattern = [1, 1, 1, 1, 1, 1, 1, 1] # setiap ketukan

# Melodi sederhana
melody_notes = [261, 294, 330, 349, 392, 440, 494, 523]

# Isi setiap beat
for beat in range(min(total_beats, len(kick_pattern)*2)):
    start = beat * samples_beat
    pat_idx = beat % len(kick_pattern)

    kick_s = make_kick(SR, beat_duration)
    snare_s = make_snare(SR, beat_duration)
    hihat_s = make_hihat(SR, beat_duration)

    if kick_pattern[pat_idx]:
        track_kick[start:start+samples_beat] += kick_s
    if snare_pattern[pat_idx]:
        track_snare[start:start+samples_beat] += snare_s
    if hihat_pattern[pat_idx]:
        track_hihat[start:start+samples_beat] += hihat_s * 0.3

    # Bass dan melodi
    note_idx = beat % len(melody_notes)
    freq_mel = melody_notes[note_idx]
    t_beat = np.linspace(0, beat_duration, samples_beat,
endpoint=False)

    # Bass note
    bass_note = 0.5 * np.sin(2 * np.pi * (freq_mel/2) * t_beat)
    bass_env = np.exp(-5 * t_beat)
    track_bass[start:start+samples_beat] += bass_note * bass_env

    # Melody note
    mel_note = 0.3 * np.sin(2 * np.pi * freq_mel * t_beat)
    mel_env = np.exp(-3 * t_beat)
    track_melody[start:start+samples_beat] += mel_note * mel_env

# --- MIXER CONFIG ---
mixer_config = {
    'Kick Drum' : (track_kick, 0.8), # (audio, volume)

```

```

'Snare Drum' : (track_snare, 0.6),
'Hi-Hat'      : (track_hihat, 0.4),
'Bass Line'   : (track_bass, 0.7),
'Melody'      : (track_melody, 0.5),
}

# --- MIXING ---
mixed_output = np.zeros(total_samples)

for name, (track, gain) in mixer_config.items():
    mixed_output += track * gain

# Normalisasi final mix
mixed_output = mixed_output / np.max(np.abs(mixed_output)) * 0.9

# --- VISUALISASI MIXER ---
fig, axes = plt.subplots(len(mixer_config)+1, 1, figsize=(14, 18))
time_arr = np.linspace(0, DURATION, total_samples)

colors_mix =
['#F44336', '#FF9800', '#FFEB3B', '#4CAF50', '#2196F3', '#9C27B0']

for idx, ((name, (track, gain)), color) in enumerate(
    zip(mixer_config.items(), colors_mix)
):
    axes[idx].fill_between(time_arr, track*gain, alpha=0.7,
color=color)
    axes[idx].plot(time_arr, track*gain, color=color, linewidth=0.8)
    axes[idx].set_title(
        f"Track: {name} (Volume: {gain*100:.0f}%)",
        fontweight='bold'
    )
    axes[idx].set_ylabel("Amplitudo")

axes[-1].plot(time_arr, mixed_output, color='#212121', linewidth=0.7)
axes[-1].fill_between(time_arr, mixed_output, alpha=0.4,
color='gray')
axes[-1].set_title("Final Mixed Output (Semua Track Digabung)",
                    fontsize=12, fontweight='bold', color='darkred')
axes[-1].set_ylabel("Amplitudo")
axes[-1].set_xlabel("Waktu (detik)")

plt.suptitle("Simple Audio Mixer - Beat Pattern 120 BPM",
            fontsize=15, fontweight='bold', y=1)
plt.tight_layout()
plt.show()

# --- PUTAR TRACK & HASIL MIX ---

```

```

print("\n" + "="*55)
print("    DENGARKAN SETIAP TRACK DAN HASIL MIX")
print("="*55)

for name, (track, gain) in mixer_config.items():
    track_norm = track * gain
    if np.max(np.abs(track_norm)) > 0:
        track_norm /= np.max(np.abs(track_norm)) * 1.1
    fname = f"15_{name.replace(' ', '_').lower()}.wav"
    save_audio(track_norm, SR, fname)
    play_audio(track_norm, SR, f"Track: {name}")

print("\n  Final Mix:")
save_audio(mixed_output, SR, "15_final_mix.wav")
play_audio(mixed_output, SR, "Final Mixed Output (Semua Track)")

```